

Review article

A survey on code smells detection using machine learning techniques

Djamel Mesbah^{a,b,c,*}, Nour El Madhoun^{c,d}, Khaldoun Al Agha^b, Anis Zouaoui^a^a Adservio, 5 Rue Bellini, Puteaux, France^b Université Paris-Saclay, CNRS, Laboratoire Interdisciplinaire des Sciences du Numérique, Gif-sur-Yvette, France^c LISITE Laboratory, Isep, 10 Rue de Vanves, Issy-les-Moulineaux, France^d Sorbonne Université, CNRS, LIP6, 4 Place Jussieu, Paris, France

ARTICLE INFO

Dataset link: <https://github.com/Kheims/DETECT>

Keywords:

Code smells
Static analysis
Machine learning
Deep learning
Software quality

ABSTRACT

Code smells are widely used indicators of deteriorating design and maintainability and their detection has evolved from heuristic and metric-based rules to learning-based approaches that operate on structural and textual views of source code. Despite this evolution, existing surveys capture only parts of the landscape and provide limited insight into how modern Machine Learning and Deep Learning techniques, code representations and datasets interact in practice. This study offers a comprehensive synthesis of code smell detection research from 2013 to March 2025, covering heuristic, evolutionary, traditional Machine Learning and Deep Learning methods. We analyze detection techniques, the representations of code on which they rely, the datasets and benchmarks used for evaluation and the extent to which prior work supports reproducibility through available tools, code and replication packages. Building on this review, we carry out a comparative study on the MLCQ dataset in which 17 models from three families (classical ML on object-oriented metrics, sequence-based DL on token sequences and graph neural networks on abstract syntax trees) are evaluated on four smells (Feature Envy, Long Method, Blob and Data Class) under a unified pipeline. The results show that no single family dominates: GNNs excel on class-level smells (Blob, Data Class), sequence models lead on Long Method and Feature Envy remains an open challenge for all families due to the lack of inter-class context in current representations. The survey consolidates current knowledge on code smell detection and identifies open challenges related to dataset coverage, annotation quality and reproducibility, while the comparative study provides a transparent baseline that can inform and be extended by future evaluations.

Contents

1.	Introduction	2
2.	Related work	2
3.	Methodology	4
3.1.	Inclusion and exclusion criteria	4
3.2.	Literature search strategy	4
3.3.	Data extraction	4
3.4.	Study quality assessment	5
4.	Background	5
4.1.	Code smells	5
4.1.1.	Impact of code smells on software quality	6
4.2.	Source code representations for smell detection	7
5.	Code smells datasets	8
5.1.	Datasets for code smell detection	8
5.2.	The MLCQ dataset	9
5.3.	The DACOSX dataset	9
5.4.	Observed challenges	9
6.	Code smell detection techniques : An overview	10

* Corresponding author at: Université Paris-Saclay, CNRS, Laboratoire Interdisciplinaire des Sciences du Numérique, Gif-sur-Yvette, France.

E-mail addresses: djamel.mesbah@universite-paris-saclay.fr, djamel.mesbah@adservio.fr (D. Mesbah), nour.el-madhoun@isep.fr (N. El Madhoun), alagha@lisen.fr (K. Al Agha), anis.zouaoui@adservio.fr (A. Zouaoui).<https://doi.org/10.1016/j.infsof.2026.108242>

Received 7 December 2025; Received in revised form 16 June 2026; Accepted 18 June 2026

Available online 24 June 2026

0950-5849/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

6.1.	Heuristic and rule-based approaches	10
6.2.	Evolutionary algorithms	10
6.3.	Metric based ML models	11
6.4.	DL techniques	11
6.4.1.	Structural models	11
6.4.2.	Pretrained models/LLMs	12
6.5.	Transfer learning for code smell detection	13
7.	Summary of findings	13
8.	Benchmark	13
8.1.	Methodology	13
8.2.	Dataset preparation	14
8.3.	Results and discussion	14
8.3.1.	Feature Envoy	15
8.3.2.	Long method	15
8.3.3.	Blob	15
8.3.4.	Data class	15
8.3.5.	Comparison with Madeyski and Lewowski	15
9.	Threats to validity	15
9.1.	Construct validity	15
9.2.	Internal validity	16
9.3.	External validity	17
9.4.	Conclusion validity	17
9.5.	Limitations	17
9.6.	Implications for researchers and practitioners	17
10.	Conclusion	17
	CReditT authorship contribution statement	18
	Declaration of competing interest	18
	Data availability	18
	References	18

1. Introduction

The development and maintenance of software systems are inherently challenging due to their continuous evolution, driven by defect correction, changing requirements and technological shifts. These demands, often under tight constraints, require a strong focus on maintainability and adaptability. Within this context, the early detection of code smells has emerged as a crucial practice. By identifying design deficiencies that compromise readability, extensibility or structure, developers can take proactive and preemptive steps to improve software quality.

Code smells are not merely stylistic issues. Empirical studies have demonstrated their correlation with higher complexity, reduced cohesion and degraded abstraction, effects observed across disparate domains and system scales [1,2]. Their negative effects extend beyond internal quality metrics to include performance degradation in runtime environments [3].

Introduced by Fowler [4], the term code smell refers to symptoms in the source code that may indicate deeper design problems. Manual identification of these smells, typically performed during refactoring, is both subjective and labor-intensive [5]. To mitigate this, automated detection tools were introduced, using heuristics and static code metrics to flag common antipatterns. Tools such as DECOR [6], JDeodorant [7] and iPlasma [8] exemplify early approaches. However, the reliance on manually set thresholds and fixed rule templates often resulted in inconsistency and limited generalizability.

In response, evolutionary approaches using genetic algorithms were proposed to discover detection rules automatically, optimizing classification performance through fitness-guided evolution. Subsequently, the field shifted toward Machine Learning (ML) techniques. Initial efforts focused on tabular representations of code metrics [9], before expanding to token-based models leveraging natural language processing [10]. More recently, graph-based representations such as abstract syntax trees and control flow graphs have enabled context-aware detection using Graph Neural Networks and transformer-based encoders [11].

Although research on code smell detection has diversified significantly, the existing surveys do not fully capture this evolution. Earlier

reviews concentrate mainly on refactoring practices and traditional heuristic tools, while several influential surveys were published before the widespread adoption of deep learning. Consequently, we still lack a consolidated view of modern ML and DL techniques, a systematic comparison of the different code representations used across studies and, importantly, a clear assessment of reproducibility aspects such as the availability of tools, replication packages or executable artifacts.

These gaps motivate the present survey. Our objective is to provide a comprehensive and methodologically rigorous synthesis of code smell detection techniques, bridging traditional rule-based systems with recent learning-based approaches. Specifically, we analyze the detection techniques introduced across the literature, the underlying code representations they employ, the datasets and benchmarks that support evaluation and the degree of reproducibility enabled through publicly available artifacts. In addition to this consolidated overview, we introduce a proof-of-concept benchmark built on the MLCQ dataset [12]. Our aim is to illustrate how representative detection models can be compared under a unified and transparent evaluation protocol, highlighting practical challenges such as class imbalance, label subjectivity and the limited reproducibility of prior studies.

The remainder of this paper is organized as follows. Section 2 reviews related work and existing surveys. Section 3 details the methodology employed to collect and analyze the studied papers. Section 4 introduces the core concepts relevant to this work. Section 5 describes the datasets and resources commonly used in the domain. Section 6 provides an overview of code smell detection techniques, including heuristic, ML and DL models. Section 7 synthesizes the main findings of the survey. Section 8 presents the benchmark methodology and experimental results. Section 9 discusses threats to validity, limitations and implications. Finally, Section 10 concludes the paper and outlines directions for future research.

2. Related work

Several surveys have been conducted in recent years to synthesize the state of research in code smell detection. These contributions have helped establish taxonomies, highlight prevalent detection strategies

and trace emerging trends. While valuable, existing works differ significantly in scope, dataset coverage and methodological depth. This section reviews the most relevant surveys and outlines how our study extends or complements them.

Fontana et al. [13] conducted one of the earliest empirical studies comparing multiple tools for automatic code smell detection. Their assessment focuses on six types of smells: Duplicated Code, Feature Envy, God Class, Large Class, Long Method and Long Parameter List, across multiple versions of the GanttProject software. They evaluated four widely used detection tools: Checkstyle, PMD, JDeodorant and inFusion, analyzing the consistency of detection results, the relevance of detected smells in practice and how these smells evolve across software versions. A key finding of their study is the low agreement between tools, particularly for smells like Feature Envy and Long Method, suggesting that different tools encode detection heuristics in significantly divergent ways. Additionally, the study showed that smells detected by tools are often removed by developers in subsequent versions, indicating their practical relevance. However, the work remains limited to heuristic-based approaches and does not explore ML- or data-driven techniques, nor does it propose any unified framework for cross-method comparison.

In parallel to these empirical comparisons of individual tools, Rasool and Arshad [14] presented a broad review of 46 primary studies on code smell mining techniques and tools published between 1999 and 2015. Their work focuses on approaches that detect Fowler's smells directly in source code and classifies them into several families, including metrics-based, symptoms-based, visualization-based, probabilistic, search-based and cooperative techniques. The survey provides an extensive comparison of detection tools, the types of smells they cover, the analysis methods employed and the programming languages and systems targeted. A key contribution of this review is its detailed analysis of the disparity between tools when applied to the same systems, which the authors attribute mainly to inconsistent smell definitions, metric thresholds and the lack of common benchmark systems. However, the survey predates the widespread adoption of deep learning techniques in this domain, does not cover token or graph-based representations and focuses primarily on heuristic and metrics-driven tools rather than learning-based models.

Lacerda et al. [15] conducted a tertiary systematic review synthesizing 20 years of research on code smells and refactoring challenges. Their work aggregates findings from prior secondary studies, with a specific focus on how code smells are defined, perceived by developers and operationalized in refactoring tools. Rather than examining detection algorithms directly, the review concentrates on conceptual issues such as ambiguity in smell definitions, inconsistencies in refactoring guidelines and the lack of consensus regarding the practical relevance of certain smells. Although their analysis provides valuable insights into high-level challenges surrounding the identification and interpretation of smells, it does not cover ML or DL-based detection techniques, nor does it analyze datasets, code representations or reproducibility aspects. As such, it offers important contextual understanding but does not address the algorithmic and empirical dimensions that are central to recent advances in automated smell detection.

Building on the ML-based perspective, Azeem et al. [16] focused exclusively on techniques that apply ML to code metrics for code smell detection. From an initial pool of over 15,000 documents, a rigorous filtering and quality assessment process led to a final selection of 15 primary studies published between 2005 and 2017. The authors placed strong emphasis on methodology, detailing their search strategies, inclusion and exclusion criteria and inter-rater reliability (Krippendorff's $\alpha = 0.98$). The study categorizes ML models into five families: decision trees (J48, JRip), ensemble learners (primarily Random Forest), neural networks (MLP), Bayesian learners (Naive Bayes) and SVM-based methods. Commonly studied smells include God Class, Long Method, Feature Envy and Data Class, with metrics such as LOC, NOM, LCOM, WMC, CC and DIT being the most prevalent. While most

studies used small-scale datasets, typically open-source Java projects like Qualitas Corpus, a few included industrial data. Notably, the study critiques the widespread use of potentially biased validation strategies, especially k-fold cross-validation without repeated trials, which may compromise the reliability of performance claims. Their methodological rigor is noteworthy, although the study remains limited to classical ML models and relatively constrained datasets.

In a more recent contribution, Yadav et al. [17] conducted a systematic literature review of 42 primary studies on ML-based code smell detection, covering publications from 2005 to 2024. Their analysis focuses exclusively on approaches that use software metrics as input features. ML algorithms are categorized into six families, with decision trees (64%), support vector machines (52%) and ensemble methods (50%) emerging as the most prevalent. God Class, Long Method, Feature Envy and Data Class are the most frequently studied smells, with over 30 different smells examined in total. The authors cataloged more than 290 software metrics across the surveyed studies, highlighting LOC, WMC, LCOM and cyclomatic complexity as the most discriminative for smell detection.

The majority of experiments relied on datasets such as Xerces, ArgoUML and Eclipse, with 39 out of 42 studies focusing exclusively on Java code. Only four studies reported the use of private industrial datasets and the authors note a general lack of programming language diversity. In terms of evaluation, accuracy, precision, recall and F1 score were the most commonly reported metrics; however, the survey underscores that accuracy can be misleading in the context of class imbalance. While this work provides a thorough synthesis of metric-based ML techniques, it does not examine token-level or structural representations, nor does it propose a unified benchmark for empirical comparison.

In their study [18], Zhang et al. conducted a comprehensive survey of 86 papers published between 2010 and 2023 on supervised learning techniques for code smell detection. Their review spans both traditional ML models applied to structural features, such as code metrics and more recent DL approaches operating on tokenized source code. The study is structured around seven research questions covering the main stages of the ML pipeline, including dataset construction, preprocessing, feature selection, model types and evaluation strategies. Decision Trees, Random Forests, SVMs and Naive Bayes are the most frequently employed models, with a growing shift toward DL techniques such as CNNs, LSTMs and ResNets observed after 2019. The smells most commonly addressed include Feature Envy, Long Method, God Class and Data Class, with many studies aiming to detect multiple smells concurrently. Nevertheless, multi-label classification and severity-level prediction remain underexplored areas. While conventional metrics like LOC, coupling and cyclomatic complexity are widely adopted, only 25% of studies incorporate feature selection and the use of semantic features is rare. Most datasets are either open-source or manually labeled using heuristic tools, with no studies leveraging industrial datasets. Moreover, class imbalance persists as a widespread and often under-addressed challenge. Although this survey encompasses both classical and DL methods, it does not propose a reproducible evaluation framework.

Taken together, these surveys provide a fragmented yet complementary view of the field. Early work such as Fontana et al. [13] and the technique-oriented review by Rasool and Arshad [14] emphasize tool support and heuristic or metrics-based detection, while the tertiary study by Lacerda et al. [15] focuses on conceptual issues such as smell definitions, refactoring practices and perceived relevance. More recent ML-oriented surveys by Azeem et al. [16], Yadav et al. [17] and Zhang et al. [18] extend the picture to classical ML and some DL techniques, but concentrate almost exclusively on metric-based representations and do not analyze in depth code representations, datasets or reproducibility practices across studies. Among the recurring challenges they highlight, the absence of a unified evaluation framework, the reliance on heterogeneous and often small-scale Java datasets and the pervasive

issue of class imbalance are central. Additionally, semantic features, structural or graph-based representations and multi-label formulations of smell detection remain underexplored.

This lack of methodological rigor is not limited to survey work but reflects a broader reproducibility problem in the field. A recent study by Lewowski et al. [19] found that only 22% of papers from 2018–2019 provided a complete reproduction package and that model descriptions were often inadequate. Furthermore, many studies rely on pre-existing datasets without clearly documenting their construction or selection rationale.

In contrast to these limitations, our survey not only synthesizes recent progress, including DL, graph-based models and large language model approaches, but also analyses code representations, datasets and reproducibility aspects in a unified manner. In addition, we carry out an exploratory comparative study based on the MLCQ dataset, in which representative models from different families are evaluated under a common experimental protocol. This study provides a transparent baseline for comparing detection techniques under consistent conditions and can be extended toward more comprehensive community-driven evaluations of code smell detection methods.

3. Methodology

In this survey, we aim to identify, categorize and analyze publications that propose or evaluate automatic code smell detection techniques, with a particular emphasis on approaches involving ML and DL. To ensure methodological rigor, we followed a structured selection process inspired by established guidelines for systematic reviews in software engineering. In particular, we adopted the PRISMA 2020 reporting principles, which organize the review process into four stages, identification, screening, eligibility and inclusion and require transparent documentation of the flow of studies through each stage [20]. The review was not pre-registered and no formal protocol was prepared; the search strategy, eligibility criteria and data extraction procedures were defined before the search was conducted and were not amended during the review.

While our primary focus is on learning-based methods, we deliberately include a selection of representative heuristic and evolutionary approaches that have played a pivotal role in shaping the evolution of code smell detection. These studies are essential to contextualize recent ML and DL-based work within the broader trajectory of detection paradigms.

3.1. Inclusion and exclusion criteria

We **included** only primary studies that satisfy all of the following criteria:

- The paper proposes, applies or evaluates an automated code smell detection technique (and not solely a smell taxonomy, refactoring catalog or empirical study of smells without detection).
- The detection targets general-purpose code smells in application source code and is not restricted to highly domain-specific categories such as test smells, security smells or infrastructure smells. Security smells, in particular, target distinct quality attributes (confidentiality, integrity), rely on different detection methods (taint analysis, vulnerability patterns) and are studied by a largely separate research community; including them would broaden the scope beyond the structural design concerns addressed by Fowler's catalog [4].
- The focus is on detection in source code artifacts (for example classes, methods, files) rather than in design models or UML diagrams.
- The technique relies on heuristic rules, code metrics, evolutionary search, ML or DL models.

- The smell types under study are clearly defined in the paper or traceable to well-established definitions.
- The paper is peer-reviewed and published in an international journal or conference (excluding extended abstracts, posters and short papers shorter than four pages).

We **excluded** papers that:

- Discuss code smells only at a conceptual level without proposing or evaluating a concrete detection method.
- Focus exclusively on severity assessment, visualization or refactoring recommendation, assuming smells as given rather than detecting them.
- Are not written in English or for which the full text was not accessible.

3.2. Literature search strategy

The literature search combined automatic queries on digital libraries with manual expansion through snowballing. We queried three main sources:

- Google Scholar,
- the ACM Digital Library,
- IEEE Xplore.

The initial search scope was set from 2010 onward. After applying the inclusion and exclusion criteria, all retained primary studies were published from 2013 onward. The final database search was conducted in March 2025. We used variations of the following search string:

```
("code smell" OR "code smells" OR "design smell" OR "design smells") AND (detection OR mining OR identification) AND (heuristic OR "code metric" OR "machine learning" OR "deep learning" OR "neural network")
```

The exact syntax was adapted to each digital library. The initial database search returned 150 records (see Fig. 1). After removing 12 duplicates and 5 entries discarded for other reasons (for example non-peer-reviewed material), 133 unique records were screened based on title and abstract. At this stage, 83 papers were excluded as clearly out of scope. Screening was conducted by the first author; the co-authors independently validated a subset of decisions and all borderline cases were discussed until consensus was reached. We retrieved the full text for the remaining 50 papers and assessed them against the inclusion and exclusion criteria described above. This led to the exclusion of 22 papers, predominantly because they did not focus on automatic detection ($n = 10$) or targeted domain-specific smells such as test or security smells ($n = 12$).

To mitigate the risk of missing relevant studies, we complemented the database search with backward and forward snowballing [21]. For each included paper and for existing surveys on code smells, we examined the reference lists (backward snowballing) and citation lists in Google Scholar (forward snowballing). This process yielded 3 additional papers that met our inclusion criteria. In total, 31 primary studies were retained after the PRISMA identification, screening and eligibility phases.

The overall selection process follows the PRISMA 2020 guidelines and is summarized in the flow diagram shown in Fig. 1, which reports the number of records identified, screened, assessed for eligibility and finally included, together with reasons for exclusion at each step.

3.3. Data extraction

For each of the 31 included studies, the first author extracted the following information into a structured spreadsheet: (i) bibliographic metadata (authors, year, venue), (ii) targeted code smells, (iii) source

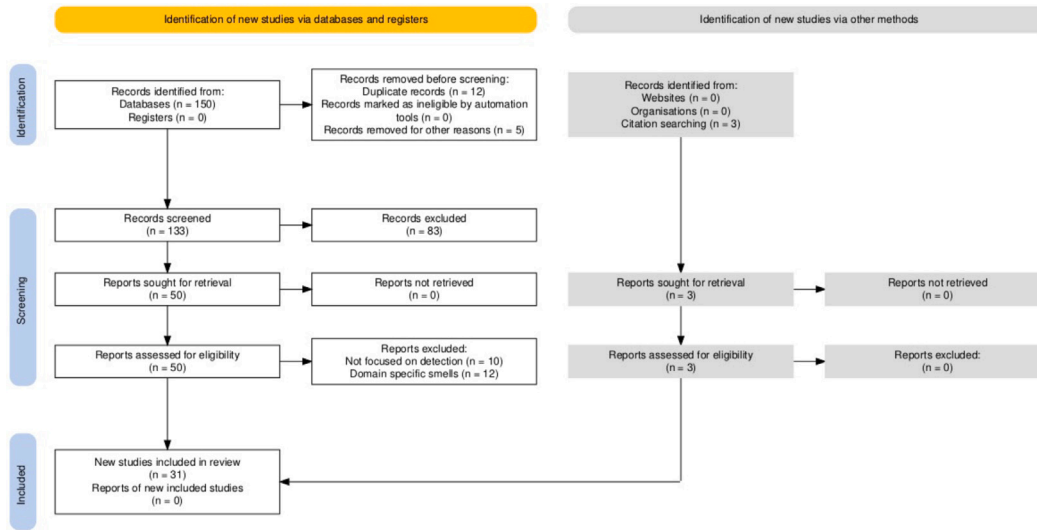


Fig. 1. PRISMA flow diagram detailing the study selection process.

code representation (metrics, tokens, AST, graph), (iv) detection technique category (heuristic, classical ML, DL, LLM), (v) dataset(s) used and their characteristics (size, language, labeling process), (vi) evaluation protocol and reported performance metrics, and (vii) availability of a replication package. These items were chosen to address the objectives stated in Section 3 and map directly to the descriptive columns of Table 1. The extracted data were reviewed by the co-authors during manuscript preparation; however, no formal independent re-extraction was performed, which is acknowledged as a limitation in Section 9.

3.4. Study quality assessment

To assess the quality of the included studies, we adopted a lightweight rubric comprising five criteria:

- **Q1:** Are the code smells being detected clearly defined or traceable to established definitions?
- **Q2:** Are the independent and dependent variables (for example code metrics, labels, evaluation measures) explicitly reported?
- **Q3:** Is the detection technique or model architecture described in sufficient detail to be understood and, in principle, re-implemented?
- **Q4:** Are the evaluation design and performance metrics clearly documented?
- **Q5:** Is an implementation (tool or code) publicly available?

Each criterion was scored as *Yes* (1), *Partially* (0.5) or *No* (0), except Q5 which was weighted double to reflect the importance of reproducibility (*Yes* = 2, *Partially* = 1, *No* = 0). The assessment was performed by the first author; borderline scores were discussed with the co-authors. Table 1 reports the availability of code (column *Code*), the handling of class imbalance (column *Imbalance*) and whether the experimental setup is well-specified (column *Defined*), which together serve as observable proxies for Q3–Q5. All 31 retained studies scored at least 2 points on this rubric; studies that fell below this threshold during the eligibility phase were excluded, as noted in Section 3.2.

Details of the selected studies are summarized in Table 1. The *Dataset* column indicates whether the dataset used comprises open-source projects such as ArgoUML, Xerxes, the Qualitas Corpus or any other publicly available software engineering dataset repurposed for code smell detection. The *Approach* column categorizes each method as heuristic, evolutionary algorithm-based (EA), ML or DL. The *Defined* column assesses whether the study presents a well-specified experimental setup, including the code smells under investigation, the metrics

or features used and the main preprocessing and modeling steps. The *Model* column lists the specific models or techniques applied. The *Language* column specifies the programming language targeted in the study. The *Code* column indicates whether an implementation is publicly available, supporting reproducibility. Finally, the *Imbalance* column reports whether the study explicitly discusses or addresses the issue of class imbalance, a known challenge in code smell datasets.

4. Background

4.1. Code smells

Taxonomy

Code smells are surface indicators of deeper structural or design issues in a software system. The concept was introduced by Fowler [4], who described smells as “symptoms” in code that often indicate the need for refactoring. Since then, a range of smells have been identified and categorized according to the level of abstraction they affect. Empirical studies have associated their presence with poor maintainability, increased fault-proneness and reduced comprehension [2,52], although this relationship varies across smell types, datasets and project domains [2].

Types of Code Smells

The literature broadly classifies code smells into three hierarchical categories based on the granularity and scope of the code element they affect [53]:

- **Implementation-level smells [54]:** These occur at the method or class level and typically involve poor programming practices or bloated code. Common examples include:
 - Long Method: a method that is too large and difficult to understand.
 - Data Class: a class with only fields and accessors but no meaningful behavior.
 - Duplicate Code: redundant logic replicated across different parts of the codebase.
- **Design-level smells [55]:** These affect class relationships or internal structure, often violating object-oriented design principles. Examples include:
 - God Class: a class that centralizes too much functionality.

Table 1
Summary of reviewed articles.

Article	Dataset	Approach	Defined	Model	Language	Code	Imbalance
ART1 [22]	Open-Source Projects	ML	No	J48, Jrip, LibSVM, Naive Bayes, RF, SMO	Java	No	No
ART2 [23]	Open-Source Projects	Evolutionary Algorithm (EA)	Yes	Parallel search, Genetic programming	Java	No	No
ART3 [24]	Open-Source Projects	ML	Yes	Rule-based	Java	No	No
ART4 [25]	Open-Source Projects	Heuristic	Yes	Infusion, JDeodorant, JSpiRIT, PMD	Java	No	No
ART5 [26]	Open-Source Projects	DL	Yes	Multilayer Perceptron (MLP)	Java	No	No
ART6 [27]	Open-Source Projects	Heuristic	Yes	Rule-based	C#, Java	No	No
ART7 [28]	Open-Source Projects	ML	Yes	SVM	Java	No	No
ART8 [29]	Open-Source Projects	ML	Yes	RF, SVM	Java	No	No
ART9 [30]	Open-Source Projects	Heuristic	Yes	Rule-based	Java	No	No
ART10 [31]	Open-Source Projects	Heuristic	Yes	Rule-based	Java	No	No
ART11 [32]	Open-Source Projects	ML	Yes	AdaBoost, Ensemble learning	Java	No	No
ART12 [33]	Open-Source Projects	DL	No	CNN, RNN	C#, Java	No	No
ART13 [34]	MLCQ	ML	No	Decision Tree, RF	Java	No	No
ART14 [35]	Open-Source Projects	ML	Yes	K-means, PCA	Java	No	No
ART15 [36]	Open-Source Projects	ML, EA	Yes	DT, SVM, XGBoost	Java	No	Yes
ART16 [37]	Open-Source Projects	Heuristic	Yes	Whale optimization algorithm	Java	No	No
ART17 [38]	Open-Source Projects	DL, GNNs	Yes	GNN, LSTM	Java	No	No
ART18 [39]	Open-Source Projects	DL	Yes	CNNs	Java	No	Yes
ART19 [40]	Open-Source Projects	DL, ML	Yes	AdaBoost, Bagging, CNN, MLP, XGBoost	Java	No	No
ART20 [41]	Not mentioned	ML	Yes	Ensemble methods	Java	No	Yes
ART21 [42]	Open-Source Projects	ML	No	SVM	Java	No	No
ART22 [43]	Open-Source Projects	ML	Yes	DT, K-NN, MLP, SVM, XGBoost	Java	No	Yes
ART23 [44]	Open-Source Projects	DL	Yes	CodeBERT, CodeGPT, CodeT5	Java	Yes	No
ART24 [45]	Open-Source Projects	Heuristic	Yes	Rule-based	C++	No	Yes
ART25 [46]	Open-Source Projects	ML	Yes	DT, K-NN, RF, SVM, XGBoost	Java	Yes	No
ART26 [47]	Open-Source Projects	DL	Yes	Bi-LSTM, GRU	Java	No	Yes
ART27 [48]	Open-Source Projects	DL	Yes	Neural network	C#, C++, Java, Python	No	No
ART28 [49]	MLCQ	DL, GNNs	Yes	GNN, Pooling strategy	Java	Yes	Yes
ART29 [10]	MLCQ	DL	Yes	Bi-LSTM, Attention	Java	Yes	Yes
ART30 [50]	MLCQ	DL	Yes	code2vec, code2seq, CuBERT	Java	No	Yes
ART31 [51]	Open-Source Projects	DL	Yes	CNN, GRU, SVM	Java	Yes	Yes

- Feature Envy: a method that interacts more with another class's data than its own.
- Inappropriate Intimacy: excessive coupling between two classes.
- Architectural-level smells [56]: These span across modules, packages or components and relate to system-wide structural degradation. Though harder to detect automatically, they are considered especially harmful. Examples include:
 - Cyclic Dependencies: modules depend on each other circularly, reducing modularity.
 - Scattered Functionality: logically related behavior is spread across the system.
 - God Component: an architectural analogue to the God Class at the system level.

A more comprehensive definition of the most prominent code smells is provided in Table 2.

4.1.1. Impact of code smells on software quality

The presence of code smells has been extensively studied for its impact on various software quality attributes, including maintainability, performance, fault-proneness and developer productivity. While some

studies emphasize the critical need to detect and refactor smells early, others reveal nuanced or even conflicting conclusions, depending on smell types, dataset characteristics and project domains.

Maintainability and evolution effort. A large body of empirical research has shown that code smells are strongly associated with reduced maintainability and increased evolution effort. Systems with high concentrations of smells such as *God Class*, *Feature Envy* or *Long Method* tend to exhibit higher change and defect-proneness, longer modification times and more complex change sets during maintenance activities [52,58]. Smelly components often accumulate technical debt, making it harder for developers to understand the underlying design intent, localize changes and safely propagate modifications across dependent modules. Over time, this leads to higher maintenance costs, more frequent regressions and reduced ability to accommodate new features and requirements, particularly in large industrial systems.

Performance degradation in resource-constrained environments. In mobile and embedded systems, performance-related smells can have a direct and measurable impact on system responsiveness and memory usage. For example, an empirical study on Android applications demonstrated that the removal of specific smells such as Internal Getter/Setter and Member Ignoring Method led to improvements of up to 12.4% in UI frame time and 3.6% in memory efficiency [3]. These findings highlight that smells are not merely maintainability concerns but may compromise runtime performance, a critical factor in environments with constrained CPU, memory and battery resources.

Table 2

Common code smells and their definitions, grouped by abstraction level and scope. Definitions follow Fowler [4]; the grouping is adapted from Mäntylä and Lassenius [57].

Level	Scope	Code smell	Definition
IMPLEMENTATION	METHOD	Long Method	A method with too many lines or variables, reducing readability and maintainability.
		Long Parameter List	A method requiring many parameters, reducing clarity and increasing coupling.
		Switch Statements	Overuse of switch or if-else chains instead of polymorphism.
	CLASS	Comments	Comments that compensate for unclear code instead of improving it.
		Duplicate Code	Identical or near-identical code fragments appearing in multiple locations.
		Primitive Obsession	Excessive reliance on primitives rather than dedicated small domain classes.
DESIGN	METHOD	Speculative Generality	Abstractions introduced “just in case” but rarely used.
		Temporary Field	A field used only under specific conditions, leading to unclear object state.
	CLASS	Feature Envy	A method that accesses more data from another class than from its own.
		Message Chain	A sequence of chained calls that exposes the internal structure of several objects.
		Alternative Classes with Different Interfaces	Classes with similar responsibilities but inconsistent interfaces.
		Data Class	A class that stores data but provides little or no behavior.
		Data Clump	A group of variables that frequently co-occur and should be encapsulated.
ARCH.	CLASS	God Class (Blob)	A class centralizing unrelated responsibilities and data.
		Inappropriate Intimacy	A class that depends too heavily on internal details of another class.
		Incomplete Library Class	A library lacking needed functionality, forcing developers to reimplement it.
		Large Class	A class that has grown excessively in size, signaled by numerous fields, methods and lines of code.
		Lazy Class	A class that contributes little and adds unnecessary complexity.

Program comprehension and developer cognitive load. Several studies have investigated the influence of code smells on developer understanding and effort. While isolated smells may not drastically impair comprehension, their combination such as the coexistence of Blob and Spaghetti Code significantly increases developer workload and decreases accuracy in comprehension tasks [59]. This suggests that developers can manage isolated anomalies, but smell accumulation leads to cognitive overload, potentially contributing to aging codebases and increased defect introduction.

Change-proneness and fault risk. A consistent finding across the literature is that smelly classes change more frequently and are more fault-prone than clean ones. This holds even after controlling for class size, suggesting that it is the presence of design flaws rather than mere volume that drives instability [52]. Structural changes are disproportionately concentrated in smelly classes, which poses long-term challenges for testing and regression safety.

Longitudinal evolution and smell persistence. Beyond snapshot analyses, studies examining long-term system evolution reveal that smells tend to persist across releases, with certain smells like God Class and Shotgun Surgery showing recurring patterns over time [60]. These smelly entities often require larger modifications and are involved in more complex changes, implying greater maintenance cost. Interestingly, some smells that are expected to discourage modification due to their entangled nature, such as Shotgun Surgery, are still frequently changed perhaps because they are central to system functionality.

Conflicting and context-sensitive impacts. A systematic literature review of over 70 studies found that while many code smells particularly God Class, Duplicate Code and Feature Envy correlate with lower quality, the effect is not uniform across all smells or projects [2]. For instance, Long Method and Long Parameter List show mixed or even positive associations in some contexts. Factors such as dataset size, programming language and domain specificity heavily influence the observed impact. These inconsistencies highlight the importance of contextualizing smell detection and avoiding one-size-fits-all assumptions in automated analysis.

4.2. Source code representations for smell detection

The choice of source code representation plays a crucial role in how effectively code smells can be detected, particularly in ML and DL settings. Different representations capture different semantic and structural aspects of code, leading to varied strengths and limitations depending on the type of smell being targeted. Broadly, the literature distinguishes between four major representation paradigms:

Metric-based representations (Tabular view). Early and still widely used approaches rely on hand-engineered code metrics that abstract object-oriented code into structured numerical features. These include size metrics such as LOC, NOM, complexity metrics cyclomatic complexity, WMC, cohesion (LCOM), coupling (CBO), inheritance (DIT, NOC) and others (see Table 3).

This representation transforms classes or methods into rows in a tabular dataset, where each metric serves as a feature. It enables the use of classical ML classifiers such as Random Forests, SVMs or Decision Trees, Fig. 2 shows an example of a Java method alongside its associated metrics.

Token-based representations (Textual view). A more recent and language-model-driven perspective treats source code as sequences of tokens identifiers, keywords, operators and literals much like words in natural language. This representation underpins the use of NLP models such as LSTMs, Transformers and pretrained language models such as CodeBERT [10].

Token-based views can either preserve exact lexical forms or abstract them via anonymization or subtokens. These models capture local syntactic and semantic patterns that can reveal poor coding practices or stylistic anomalies (see Fig. 3).

Graph-based representations (Structural view via ASTs). Another class of representations leverages the inherent tree or graph-like structure of code. Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs) [61] and Program Dependency Graphs (PDGs) are the most prominent.

Such structures are particularly effective for detecting smells that depend on control logic, type relationships or structural violations such

Table 3
Software metrics commonly used in code smell detection, grouped by category.

Category	Metric	Full Name	Definition
SIZE	LOC/CLOC	Lines of Code	Total number of non-blank, non-comment lines in a class.
	MLOC	Method Lines of Code	Number of non-blank, non-comment lines in a method.
	NOM	Number of Methods	Count of methods defined in a class.
	NOF	Number of Fields	Count of fields (instance variables) declared in a class.
	NOP	Number of Parameters	Count of formal parameters in a method signature.
COMPLEXITY	WMC	Weighted Methods per Class	Sum of cyclomatic complexities of all methods in a class.
	VG (CC)	McCabe's Cyclomatic Complexity	Number of linearly independent paths through a method.
	AMW	Average Method Weight	Average cyclomatic complexity per method (WMC/NOM).
	MCC	Method Calling Coupling	Longest chain of method calls from a method.
COUPLING	CBO	Coupling Between Objects	Number of classes to which a class is coupled.
	ATFD	Access to Foreign Data	Number of attributes from unrelated classes accessed directly.
	FDP	Foreign Data Providers	Number of distinct classes whose attributes are accessed.
COHESION	LCOM	Lack of Cohesion in Methods	Degree to which methods in a class access different subsets of fields.
	TCC	Tight Class Cohesion	Ratio of directly connected method pairs sharing field access.
	LAA	Locality of Attribute Access	Ratio of own attributes accessed to total attributes accessed.
	WOC	Weight of Class	Ratio of non-accessor public methods to all public methods.
INHERITANCE	DIT	Depth of Inheritance Tree	Length of the longest path from a class to the root of the hierarchy.
	NOC/NSC	Number of Children/Subclasses	Number of immediate subclasses of a class.
	BOvR	Base-class Overriding Ratio	Ratio of overridden methods to total inherited methods.
	NProtM	Number of Protected Members	Count of protected fields and methods.
ACCESS	NOAM	Number of Accessor Methods	Count of getter and setter methods in a class.
	NOAP	Number of Public Attributes	Count of public instance variables (indicates weak encapsulation).
	IMC	Internal Method Calls	Number of method calls to methods within the same class.
	xMC	External Method Calls	Number of method calls to methods in other classes.

```

1 public class Test {
2     public int max(int a, int b) {
3         if (a > b) {
4             return a;
5         } else {
6             return b;
7         }
8     }
9 }

```

Metric	Value	Explanation
NOM	1	One method defined
NOF	0	No fields declared
CLOC	9	Non-blank lines in class
MLOC	7	Non-blank lines in method
NOP	2	Parameters: a, b
ATFD	0	No foreign data accessed
NOAM	0	No accessor methods
WOC	1.0	All methods contain logic
NOAP	0	No public attributes
VG	2	Cyclomatic complexity

Fig. 2. A simple Java class (left) and its associated design metrics (right). These metrics are commonly used in traditional code smell detection approaches and can serve as features for metric-based classifiers.

as God Class and Feature Envy. Models like Graph Neural Networks (GNNs) are applied over these structured inputs to learn contextualized representations.

To illustrate the structural richness captured by ASTs, Fig. 4 shows a simple Java method alongside its corresponding AST. Even in this small example, the AST encodes method declarations, control flow and return statements, features that structural models can exploit to detect subtle design issues.

Embedding-based representations (Learned semantics). Some recent approaches adopt embedding-based representations where models like code2vec [62], code2seq [63] or CuBERT [64] generate dense vector representations of code snippets. These embeddings are either learned from AST paths, token contexts or both, aiming to capture rich semantic patterns in a latent space.

These representations are often used in hybrid settings, for instance, concatenating code2vec embeddings with metric features or using embeddings as input to a BiLSTM or attention-based classifier [10].

5. Code smells datasets

5.1. Datasets for code smell detection

The performance and generalization of code smell detection models are deeply influenced by the datasets they are trained and evaluated on. However, the landscape of datasets in this domain remains fragmented, with inconsistencies in availability, scale, labeling procedures and pre-processing pipelines all of which hinder objective and reproducible comparisons across studies [19].

Broadly, we identify three types of datasets used in the literature:

- **Public benchmark datasets:** A limited number of curated and openly available datasets exist. Notably, the MLCQ [12] and DA-COSX [65] datasets provide labeled Java methods with multiple types of code smells and associated severity levels. These datasets are increasingly used to train and benchmark smell detection models in a reproducible manner.
- **Industrial or private datasets:** Some studies collaborate with industry partners to access proprietary codebases. These datasets

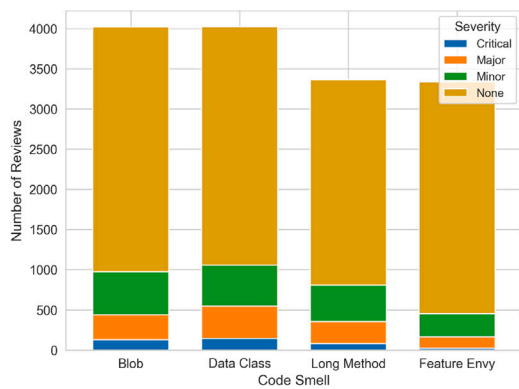


Fig. 5. Distribution of severity levels per Code Smell in the MLCQ dataset.

reported without publicly sharing the underlying data or even the full labeling procedure, preventing reproducibility and limiting the potential for replication studies or fair benchmarking.

Another pervasive challenge is the problem of class imbalance. Code smells, particularly those like Feature Envy, Middle Man or Refused Bequest, often occur far less frequently than other more prominent smells such as Long Method or God Class. This imbalance leads to datasets where the majority of samples are non-smelly and among smelly samples, a handful of types dominate. Without proper attention to this issue, models trained on such distributions may overfit or report inflated performance metrics, especially when evaluated using accuracy or naive precision without accounting for minority-class behavior. This is particularly problematic in the context of multi-label learning, where correctly identifying rare smells is crucial but under-rewarded in coarse metrics. This issue is clearly illustrated by the smell distribution in the MLCQ dataset (see Fig. 5).

On top of that we can cite the ambiguity and inconsistency in labeling strategies. Even when tools such as Designite, iPlasma or JDeodorant are used to annotate datasets, their heuristic nature and version-specific implementations result in considerable variation across studies. Two datasets using the same tool may still diverge in output due to configuration choices, smell definitions or filtering criteria. This makes it difficult to establish clear boundaries on what constitutes a positive sample. For example, in the MLCQ dataset, the average length of code samples identified as a long method is 20.7 lines, a threshold much lower than the one used in PMD [71].

Furthermore, the granularity at which smells are labeled often differs. Some datasets annotate classes, others methods and in some cases, architectural units, leading to difficulties in unifying evaluation strategies. For example, datasets like MLCQ annotate at the class level for some smells and at the method level for others, while DACOSX annotates at the method level, which offers consistency but sometimes obscures class-level or architectural-level smells, either by flattening them or by attributing them heuristically to single methods. This introduces noise and may affect the reliability of supervised learning labels.

Lastly, most datasets, including MLCQ and DACOSX, focus solely on Java codebases which raises concerns about the generalizability of models trained on them. The absence of multi-language datasets limits our understanding of whether a given model learns properties of code smells or simply exploits language-specific patterns. Similarly, the over representation of open-source projects and the underreporting of industrial or real-world systems means that conclusions drawn from existing datasets may not generalize well to large-scale or domain-specific codebases.

6. Code smell detection techniques : An overview

6.1. Heuristic and rule-based approaches

Heuristic approaches to code smell detection rely on expert-defined rules, often leveraging static software metrics or structural patterns to identify symptoms of design flaws [72]. Inspired by Fowler's catalog of code smells [54], early detection techniques defined fixed thresholds on metrics, such as marking a class as a God Class when its Weighted Method Count (WMC) exceeds 50 [26,73]. Marinescu's detection strategies further formalized this approach by combining metrics with logical operators (AND/OR) to model smells as compound rules [74]. Tools like PMD and JDeodorant integrate these metric-based heuristics and remain widely used in industry.

One of the most comprehensive rule-based systems is DECOR [6], which describes smells using a domain-specific language (DSL) and integrates naming conventions such as "Process", "Control" and structural relationships. Similarly, FindSmells [75] provides a customizable framework for rule definition. These approaches are valued for their transparency and low computational cost but depend heavily on subjective threshold tuning and domain-specific calibration. As a result, they often struggle with generalization across projects, leading to false positives or negatives, especially when code deviates from assumed architectural norms [65].

Several recent efforts have aimed to improve or evaluate these heuristic methods. For example, [27] presented a lightweight strategy using SQL queries and regular expressions to detect all 22 Fowler smells across Java and C# systems. Their method, tested on projects like JUnit and JHotDraw, emphasized flexibility through user-defined thresholds, reporting up to 97% accuracy. However, the lack of implementation transparency limits reproducibility.

The fragility and subjectivity of thresholds were also underscored in a comparative study by [25], which benchmarked four popular tools: PMD, JDeodorant, inFusion and JSPIRIT on two annotated systems. They reported wide variance in detection quality, with precision and recall ranging from 0% to 100% depending on the tool and context. Disagreements stemmed from differing metric definitions and thresholds, highlighting the need for careful tool selection based on project-specific characteristics.

To address the rigidity of fixed thresholds, hybrid approaches have emerged. Notably, EGAPSO [30] applied a metaheuristic search combining genetic algorithms and particle swarm optimization, using distance-based similarity measures rather than static cutoffs. Evaluated on multiple Java projects, EGAPSO outperformed classical methods like GA and DECOR in both precision and recall across smells such as Blob and Spaghetti Code. Similarly, [31] proposed integrating detection with refactoring validation, filtering false positives by checking the applicability of improvement strategies. Their evaluation across four systems showed improved detection reliability, particularly for smells where metric-based detection is notoriously noisy.

In summary, while heuristics remain foundational due to their simplicity and interpretability, their effectiveness is bounded by threshold sensitivity, project variability and limited contextual awareness. This has driven the field toward adaptive or data-driven methods, including machine learning and hybrid search techniques, which seek to overcome these core limitations.

6.2. Evolutionary algorithms

Another family of techniques frames code smell detection as an optimization problem, leveraging evolutionary algorithms to automatically derive detection rules. These approaches address the inherent subjectivity and difficulty of manually setting thresholds by searching the space of rule combinations for high-performing solutions. For example, Kessentini et al. [76] applied genetic programming to evolve combinations of metrics and thresholds that best distinguish smelly from

clean classes. Their fitness function prioritized maximizing detection of known smelly instances while minimizing false positives, calibrating the rules directly to the data rather than relying on human intuition.

Building on this idea, multi-objective genetic algorithms have been explored to balance competing goals such as maximizing detection rates while minimizing false alarms. Mansoor et al. [77] demonstrated how evolutionary models can search for Pareto-optimal metric combinations that capture the essence of smells without mislabeling well-designed code. Variants based on particle swarm optimization [78], artificial immune systems [79] and other meta-heuristics have also been proposed to automate and adapt smell detection across different systems.

A notable contribution is the parallel evolutionary framework proposed in [23], which combines two cooperative search strategies: one based on genetic programming evolving detection rules from structural metrics and another based on genetic algorithms modeling deviation from reference code. This Parallel Evolutionary Algorithm (P-EA) approach employs an intersection-based fitness function to build consensus between the two search strategies. Evaluated on eight widely-used Java projects, the method significantly outperformed both single-population metaheuristics and traditional metric-based techniques in precision and recall. However, the lack of code availability limits replication.

Addressing a key gap in the literature, the ADIODE method introduced in [36] focuses on code smell detection in imbalanced data settings an often-overlooked challenge. ADIODE uses an evolutionary algorithm to evolve Oblique Decision Trees (ODTs), optimizing the Area Under the Curve (AUC) as a fitness function due to its robustness in skewed class distributions. The system includes a detector generation phase, where smell-specific ODTs are evolved and an application phase, where ensembles of detectors are used with majority voting. Tested on five manually validated Java systems, ADIODE outperformed classical machine learning baselines, particularly in the presence of small disjuncts and noisy labels. The study also raised awareness of the challenges posed by multi-label ambiguity in smell annotation and advocated for tailored models and evaluation metrics in such contexts.

6.3. Metric based ML models

To overcome the rigidity of fixed-threshold heuristics, researchers have explored supervised ML models that use software metrics as input features. In this paradigm, code entities such as classes or methods are represented as fixed-length vectors of structural or object-oriented metrics (see Table 3) and a model is trained to classify them as “smelly” or “clean” using labeled data. Early studies adopted simple classifiers like Naïve Bayes and decision trees to detect smells such as God Class and Feature Envy [80,81]. Since then, a wide array of algorithms have been employed, including SVMs, k-NN and ensemble methods like Random Forests and AdaBoost [82].

Typically, datasets are manually labeled or generated using existing detection tools and classical ML models are trained to learn the mapping from metrics to labels. These models have reported high performance, up to 90% F1-scores in some cases using ensembles [9], while offering advantages such as retrainability and reduced subjectivity compared to hard-coded rules. However, their effectiveness still depends heavily on the quality of the dataset, the expressiveness of the selected metrics and the feature engineering pipeline.

Recent work has explored the impact of feature selection on both performance and efficiency. For example, [29] showed that reducing the feature set to the most discriminative metrics through techniques like mutual information can significantly improve both accuracy and runtime. Using SVM and Random Forest on public datasets, the study demonstrated that tailored subsets of object-oriented metrics outperform full metric sets in practical scenarios.

The challenge of limited or noisy training data has also driven research into more robust learners. The SEADART framework [32], for instance, employs AdaBoost to detect Refused Bequest by training on

synthetically generated examples. The authors simulated poor inheritance relationships within high-quality Open Source Java systems to augment the dataset and mitigate the scarcity of labeled examples. The ensemble approach demonstrated clear gains over classical tools and highlighted AdaBoost’s resilience to limited supervision.

In scenarios where smell annotations are absent, Anomaly detection methods have also been explored for code smell detection, particularly when specific smell labels are unavailable. One study [35] applied algorithms such as K-Means, Gaussian Mixture Models (GMM), Autoencoders, PCA and Bayesian Networks to detect smells across three Java projects: ArgoUML, Eclipse and JEdit. Feature selection was performed using Recursive Feature Elimination (RFE), though the precise features were not disclosed. K-Means clustering achieved the highest reported accuracy.

Beyond binary classification, some researchers have reframed smell detection as a regression task. In [34], the authors used over 3000 expert reviews from the MLCQ dataset to assign severity scores to code samples, reflecting human perception rather than binary ground truth. By training models like Random Forest on these scores, using metrics extracted with PMD and Understand, they outperformed rule-based tools like PMD in capturing nuanced severity, especially for borderline cases.

Some studies also tackled the previously noted challenge, namely, class imbalance. In [43], the authors addressed this issue using random oversampling and tested five classifiers (DT, SVM, k-NN, XGBoost and MLP) on datasets for four smells. XGBoost reached up to 100% accuracy on Data Class and Long Method when trained on balanced data. The study confirmed that data balancing can yield 2%–5% gains across classifiers and should be standard practice in smell detection pipelines.

Hybrid approaches have also emerged to push the boundaries of classical ML. For instance, [42] proposed combining SVMs with hyperparameter optimization (via GridSearch) and feature selection techniques (wrapper-based and mutual information). Evaluated on the Azureus Open Source project for Blob, Functional Decomposition and Swiss Army Knife, their method reported up to 99.76% accuracy.

6.4. DL techniques

6.4.1. Structural models

Leveraging Abstract Syntax Trees (ASTs) for code smell detection enables models to incorporate rich structural information inherent in the source code. ASTs capture the syntactic and hierarchical relationships between program elements, providing a fine-grained representation of code that goes beyond surface-level metrics or token sequences. This structural abstraction facilitates the detection of complex code smells, such as those involving control flow irregularities or deep nesting that may span multiple lines or involve intricate dependencies. Unlike token-based approaches, AST-based methods are often more context-aware and less sensitive to superficial variations in code style. Moreover, because ASTs represent language-agnostic syntactic constructs, they offer increased portability and adaptability across programming languages, making them a robust foundation for general-purpose smell detection.

Recent advances have explored the integration of ASTs with deep learning models, particularly Graph Neural Networks (GNNs), to learn expressive, structure-aware representations for code smell detection. Several studies have proposed methods that encode ASTs as graphs and apply GNN architectures to capture both local and global syntactic dependencies. In [11], the authors introduced AST-CSD, the first approach to explicitly use AST-based representations for multi-granularity code smell detection. Their dataset comprised 500 Java projects from GitHub, labeled using Designite, with 100 projects selected for implementation-level smells and all 500 for design-level smells. ASTs were generated using JavaLang from class and method-level fragments and paired with corresponding smell annotations for Insufficient Modularization, Deficient Encapsulation and Feature Envy.

The model architecture builds upon ASTNN [83], which recursively encodes statement-level AST subtrees before feeding them into a bidirectional GRU to capture sequential dependencies. These representations are then pooled and passed through a feedforward network for binary classification. To evaluate performance, AST-CSD was benchmarked against three token-based baselines (CNN-1D, CNN-2D and LSTM-RNN), showing statistically significant improvements across precision, recall and F1-score. Significance testing via Wilcoxon signed-rank and Cliff's delta further substantiated the robustness of the AST-based approach, though performance was influenced by smell type and class imbalance.

A complementary direction was proposed by the Hybrid Multi-level Model (HMML) in [39], which integrates both structural and lexical representations. The authors employed a Code Property Graph (CPG) to encode program structure and a Bi-LSTM with attention to process token sequences. Both streams operate in parallel, with their outputs fused through a weighted sum before final prediction via a sigmoid layer for multi-label classification. The dataset of 100 curated GitHub Java projects was again labeled using Designite.

HMML achieved consistently strong results, particularly on smells like Magic Number and Missing Default, outperforming traditional baselines such as Random Forest, which suffered from low recall. Evaluation included Win/Tie/Loss comparisons and statistical tests, confirming the hybrid model's effectiveness. Interestingly, while ASTNN proved competitive in single-label settings, HMML demonstrated robust performance across both single and multi-label tasks, emphasizing the advantage of integrating structural and token-level signals.

More recent developments in structural analysis further extend this direction by refining how ASTs are encoded as graphs. One study [49] constructs AST graphs for Java methods and employs GNNs augmented with graph pooling strategies to detect Long Method and Blob smells. To guide pooling, the authors introduce a node scoring mechanism that integrates both global and local context, leading to superior performance over clustering-based pooling. The model also addresses the previously mentioned class imbalance problem by adapting fuzzy sampling techniques from software defect prediction, which yield more stable results than SMOTE, a method ill-suited to graph data. Additionally, the use of hyperbolic space embeddings allows the model to better preserve hierarchical structures typically flattened in Euclidean space. Although the method focuses on only two smells and a single language (Java), its reproducibility and general design suggest strong potential for extension to other object-oriented languages with similar AST representations.

6.4.2. Pretrained models/LLMs

Recent advances in natural language processing have led to the development of pretrained models and large language models (LLMs) trained on vast corpora of code. These models capture both syntactic and semantic features of source code and can be fine-tuned for downstream tasks such as code smell detection. By leveraging transfer learning, pretrained models reduce the reliance on manual feature engineering and offer improved generalization across projects and smell types. This subsection reviews studies that integrate pretrained embeddings or language models, either as feature extractors or end-to-end predictors into code smell detection pipelines.

In [50], Kovačević et al. investigated the use of pretrained code embeddings in combination with conventional source code metrics for detecting God Class and Long Method smells. They evaluated multiple representations: (1) pretrained embeddings from code2vec, code2seq and CuBERT; (2) hand-crafted metrics; and (3) hybrid representations combining metrics and heuristics. Their experiments, conducted on the MLCQ dataset, also included heuristic-based baselines and extensive error analysis.

Their results showed that code2vec and code2seq embeddings lacked generalization, whereas CuBERT embeddings outperformed other approaches. Interestingly, models trained solely on code metrics

also outperformed traditional heuristic methods. CuBERT-based representations proved more robust, capturing semantic information that metric-based approaches often missed due to their fragility and lack of expressiveness.

This work was among the first to assess the impact of pretrained neural code embeddings for smell detection, highlighting how transfer learning can overcome limitations of manual feature engineering. Nevertheless, the study also underscored weaknesses in the MLCQ dataset, namely, annotation inconsistencies and lack of standardized labeling protocols. The authors advocated for adopting best practices from NLP annotation guidelines to improve dataset quality and reproducibility.

In [51], Zhang et al. proposed DeleSmell, addressing common issues in smell detection such as incomplete feature extraction and class imbalance. They built a dataset from 24 real-world projects, generating positive examples by applying automated refactorings to inject smells. Their model architecture combined CNN and GRU-Attention branches, with final classification performed via SVM. DeleSmell integrated both structural metrics from iPlasma and semantic features via word2vec and LSA, reporting up to 4.41% accuracy improvement for Brain Class and Brain Method detection.

In a more recent study [84], the authors introduced a human-in-the-loop framework for calibrating deep learning models. Using the DACOS dataset [66], they filtered and tokenized code samples with CodeBERT and CodeT5, then trained several autoencoder-based classifiers: AE-MLP, AE-LSTM and a Variational Autoencoder (VAE). The VAE, trained only on positive samples, used a reconstruction loss threshold for classification and initially outperformed other models in standalone evaluation.

However, when incorporating human feedback, the combination of AE-MLP with CodeBERT yielded the best results. The authors deployed a web-based interface to collect real-time feedback from 14 computer science professionals. This feedback loop significantly improved detection performance, with an average F1-score increase of 15.49%, varying by smell type. The study emphasized that human feedback is especially valuable for more nuanced smells and in low-data settings, offering a promising direction for hybrid AI-human detection systems.

The most recent wave of research in code smell detection has shifted toward the use of Large Language Models (LLMs) and pre-trained code representations. Inspired by the success of models such as CodeBERT, CodeT5 and GPT in software engineering tasks, these approaches leverage models trained on large-scale code corpora to identify suboptimal programming patterns.

One common strategy involves using frozen embeddings from pretrained models as input features to a classifier. For example, Kovačević et al. [85] used CodeT5 embeddings alongside traditional metrics to train a classifier. Their results showed that learned representations alone can match the performance of handcrafted features, suggesting that large models capture relevant structural and semantic information useful for identifying smells.

Another technique involves fine-tuning pre-trained models directly for the classification task. Liu et al. [86] proposed a two-stage method that aggregates intermediate representations from CodeBERT and feeds them into a Bi-LSTM. An attention layer selects the most informative signals, which are then passed to a fully connected network. This architecture achieved strong results while remaining more computationally efficient than end-to-end transformer-based fine-tuning.

In addition to embedding-based and fine-tuned approaches, some recent work explores prompting LLMs directly. Instead of training a classifier, these methods use zero or few-shot prompting to query models such as GPT-4 or LLaMA, asking whether a given snippet exhibits known code smells [87]. This approach relies on the model's broad exposure to real-world code and coding practices during pretraining.

LLM-based methods present several advantages. They capture semantic and stylistic cues that are difficult to encode using rules or metrics, such as naming inconsistencies or unconventional control flows. They are also data-efficient, requiring fewer labeled examples thanks

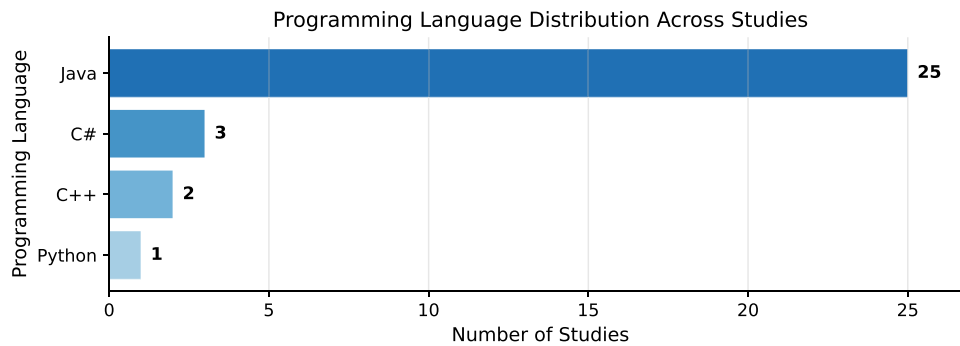


Fig. 6. Language distribution across studies.

to their pre-training. However, they also raise practical concerns. Fine-tuning and inference are resource-intensive and subjectivity in smell definitions can lead to inconsistent predictions unless the model is carefully guided.

Despite these challenges, LLMs represent a promising direction in the field. Their ability to generalize across languages, coding styles and smell types offers an opportunity to rethink how we approach software quality analysis. As the field evolves, LLM-based techniques are likely to emerge as a distinct class within the broader landscape of code smell detection.

6.5. Transfer learning for code smell detection

Most studies on code smell detection rely on datasets constructed exclusively from Java projects, whether sourced from open-source repositories, proprietary codebases or curated using static analysis tools. This language homogeneity raises concerns about the generalizability of detection techniques across diverse programming ecosystems. Fig. 6 illustrates the language distribution across the primary studies reviewed in this survey, revealing a clear dominance of Java-based datasets.

Although transfer learning has gained traction in various software engineering tasks [88,89], its application to code smell detection remains relatively unexplored. Only a limited number of studies have investigated the reuse of learned representations or models across domains, projects or programming languages. For instance, Tushar et al. [33] evaluated CNN, RNN and autoencoder architectures for smell detection and explored cross-language transfer by training models on Java and testing on C#. While performance varied, the results demonstrated the feasibility of transfer learning in this context. Similarly, Lima et al. [48] proposed a cross-project learning approach using simple MLP to study the mechanics of transfer. They extracted code smells from Java projects using DesigniteJava and from C++, C# and Python projects using Understand. Their findings showed effective knowledge transfer between Java and C#, with as few as 32 samples sufficient for many smells, though more complex cases, such as Multifaceted Abstraction, required larger training sets.

7. Summary of findings

The analysis of 31 primary studies reveals several patterns and gaps in the current state of ML-based code smell detection.

Smells. Research concentrates on four smells: Blob (God Class), Long Method, Feature Envy and Data Class. Architectural and cross-component smells are rarely addressed by learning-based methods and test or security smells are studied by separate communities with distinct techniques.

Techniques. The field has evolved from threshold-based heuristics to metric-driven classical ML and, more recently, to DL models operating on token sequences or AST graphs. Despite this progression, classical models such as Random Forests and SVMs remain competitive when paired with well-chosen OO metrics, particularly for class-level smells.

Datasets. Most studies rely on small, single-annotator datasets extracted from a handful of Java open-source projects. MLCQ and DaCoSX are among the few publicly available corpora with multi-annotator labels, but both cover only Java and a limited set of smells. No cross-language benchmark exists.

Evaluation. F1-score is the dominant metric; MCC, which is more robust to class imbalance, is reported in only a minority of the surveyed studies. Yao and Shepperd [90] showed that F1 can produce misleading rankings under class imbalance, and recommended MCC as a chance-corrected alternative. Accuracy is still used in several studies despite heavily skewed label distributions. Evaluation protocols vary widely (hold-out vs. cross-validation, stratification, threshold tuning), making cross-study comparison unreliable.

Reproducibility. Fewer than half of the surveyed studies publish their source code or trained models. When code is available, it often lacks documentation or depends on obsolete libraries, limiting practical reuse.

These observations motivate the unified benchmark presented in the next section, which evaluates representative models from three families under identical conditions on MLCQ.

8. Benchmark

8.1. Methodology

To address the lack of a unified evaluation framework identified in Section 7, we benchmark representative models from three families (classical ML, sequence-based DL and graph neural networks) on the MLCQ dataset under identical preprocessing, label definitions and evaluation protocols. All four annotated smells (Feature Envy, Long Method, Blob and Data Class) are evaluated simultaneously as a multi-label classification task (see Fig. 7 for their prevalence in the literature). The full pipeline is driven by a single YAML configuration file per experiment; source code and configurations are publicly available.¹

For each family, we select models that reflect the main approaches synthesized in Section 6:

- **Classical ML** (OO metrics): Random Forest, Decision Tree, MLP, J48 (C4.5), SVM, Naive Bayes, XGBoost and KNN trained on 21 Designite features with per-label threshold tuning and 5-fold stratified cross-validation with grid search.
- **Sequence DL** (token sequences): LSTM, GRU, BiLSTM, BiLSTM+Attention, CNN and CodeBERT trained on tokenized code sequences with focal loss and early stopping. CodeBERT fine-tunes a pretrained encoder with its own subword tokenizer.
- **GNN** (AST graphs): GCN, GAT and GraphSAGE trained on ANTLR-parsed AST graphs with Child edges using focal or weighted BCE loss.

¹ <https://github.com/Kheims/DETECT>.

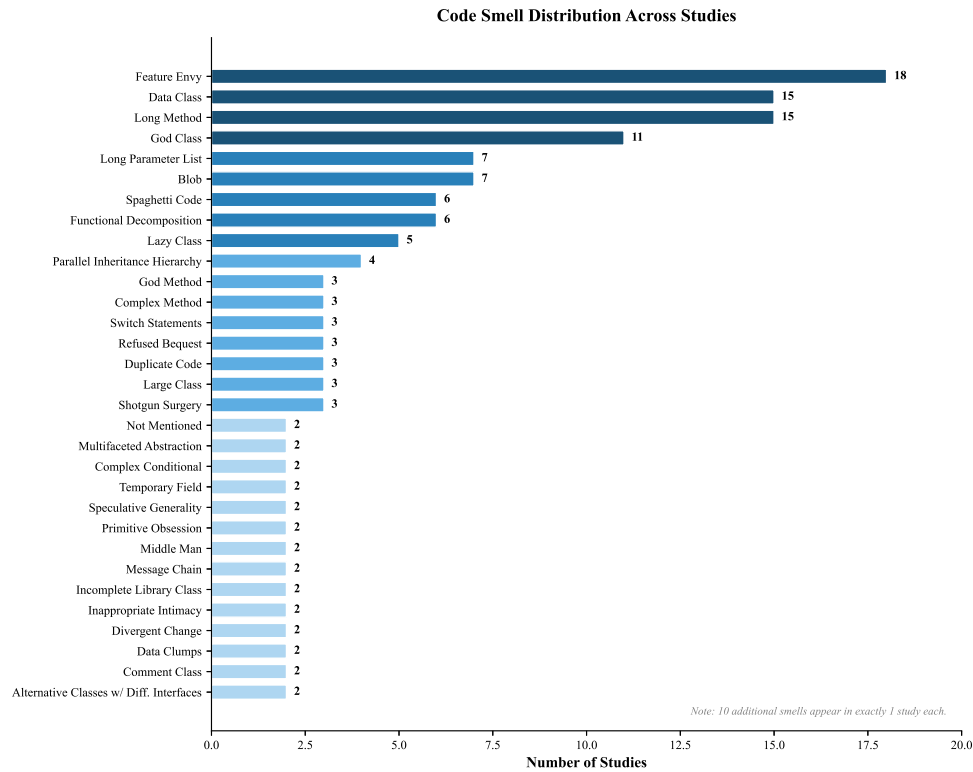


Fig. 7. Smells distribution across studies.

All experiments are repeated over 10 random seeds; we report mean $\pm$ std of F1-macro, MCC, PR-AUC and per-label F1.

8.2. Dataset preparation

The full preprocessing workflow is illustrated in Fig. 8. Starting from MLCQ metadata which provides precise commit-level information such as the repository URL, commit hash, affected lines, severity and annotated smell, we programmatically retrieve the original code from GitHub. These retrieved fragments are then consolidated into a unified JSON dataset that stores raw code snippets and binary labels. Since each code sample in MLCQ may be reviewed by multiple annotators, each assigning an independent severity level (none, minor, major or critical), an aggregation step is required before training. For each (sample, smell) pair, we collect all reviewer assessments and apply a majority-vote rule: if the majority of annotators assign a severity of minor or higher, the sample is labeled as smelly for that smell; otherwise it is labeled as non-smelly. This produces a binary label per smell type for each code sample.

This JSON dataset serves as the central intermediate format from which we derive three types of model inputs.

Classical ML features. Object-oriented design metrics are extracted via DesigniteJava for each code sample. The 11 type-level metrics capture class structure and coupling: Number of Fields (NOF), Number of Public Fields (NOPF), Number of Methods (NOM), Number of Public Methods (NOPM), Lines of Code (LOC), Weighted Methods per Class (WMC), Number of Children (NC), Depth of Inheritance Tree (DIT), Lack of Cohesion of Methods (LCOM), Fan-In and Fan-Out. Three method-level metrics (LOC, Cyclomatic Complexity and Parameter Count) are aggregated at the class level using max, sum and average, yielding 9 features; together with the method count, this produces 10 method-derived features for a total of 21. All features are standardized to zero mean and unit variance before training.

Sequence DL features. Code snippets are tokenized with a regex-based splitter that separates identifiers, keywords, operators and literals. The vocabulary is limited to the 10 000 most frequent tokens; rarer tokens are mapped to a special unknown symbol. Each sequence is truncated or padded to a fixed length of 512 tokens. A 64-dimensional Word2Vec embedding is pre-trained on the training corpus and used as the initial representation for each token.

GNN features. Each code snippet is wrapped in a synthetic class stub to form a valid Java compilation unit, then parsed with the ANTLR Java 8 grammar into an abstract syntax tree. The tree contains two kinds of nodes: SyntaxNodes corresponding to grammar rules (e.g. *methodDeclaration*, *classBodyDeclaration*, *ifThenStatement*) and SyntaxTokens for terminal symbols (keywords, identifiers, literals). The full vocabulary comprises 202 distinct node types from the Java 8 grammar. Each node is represented by a learnable 64-dimensional type embedding concatenated with 5 numeric features: a binary is-token flag, normalized start line, start column, line span and token length, yielding a 69-dimensional input vector. Nodes are connected by Child edges that follow the AST parent-child hierarchy. After 2–3 rounds of message passing, node representations are aggregated via global mean pooling into a graph-level vector, which is passed to a two-layer classifier producing the 4 binary smell predictions.

8.3. Results and discussion

Tables 4, 5 and 6 present the results across the 17 models. Following Madeyski and Lewowski [91], we use MCC as the primary per-smell metric, which is chance-corrected and more informative than F1 under class imbalance [90]. The two metrics can rank models differently because F1 considers only the positive class while MCC also accounts for true negatives: a model that predicts the positive class more aggressively can inflate F1 through high recall while accumulating false positives that erode MCC. This is visible on Blob and Data Class, where GCN ranks first by F1 but second by MCC, while GraphSAGE shows the opposite pattern. We organize the discussion by smell; detailed per-model scores are in the tables.

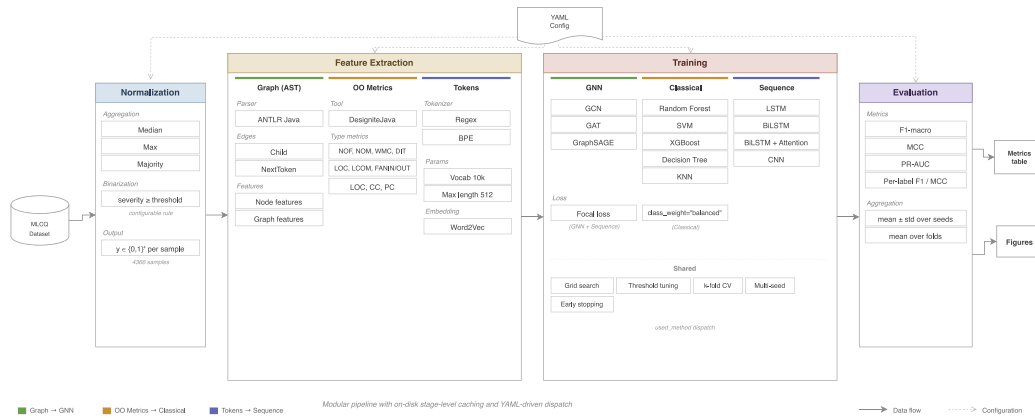


Fig. 8. Unified pipeline architecture with on-disk stage-level caching and YAML-driven dispatch. Three feature extraction paths feed into the corresponding model families.

8.3.1. Feature Envy

Feature Envy has 103 positive samples (2.4%) and is the hardest smell across all families. No model exceeds MCC 0.28. Decision trees (DT, J48) score slightly above other classical models, but all remain below 0.24. Among sequence models, BiLSTM+Attention and CodeBERT reach the highest MCC on this smell (~ 0.24 – 0.28), while LSTM collapses near zero. GNNs produce the lowest scores (MCC 0.13–0.18): single-snippet ASTs lack the inter-class coupling information that defines Feature Envy. Naive Bayes reaches near-perfect recall but with precision below 0.05 (Fig. 9), making it impractical. No model reaches a useful precision–recall balance, suggesting that detecting coupling between classes requires representations that span multiple classes.

8.3.2. Long method

Long Method (243 positives, 5.6%) is the most reliably detected smell, with several models exceeding MCC 0.7. BiLSTM+Attention leads across all families and is the only model to reach a useful precision–recall tradeoff (precision 0.66, recall 0.83). GCN and GAT approach similar MCC levels. Classical models are consistent (MCC 0.63–0.68), with XGBoost at the top. LSTM and Naive Bayes collapse on this smell for opposite reasons: LSTM produces near-random predictions, while Naive Bayes labels almost everything as positive. The token-level patterns that characterize Long Method (repetitive statements, deep nesting, long sequences) are captured well by both attention-based and graph-based models.

8.3.3. Blob

Blob (288 positives, 6.6%) is a class-level smell where structural information matters most. GNNs produce the strongest results, with GraphSAGE achieving the highest MCC across all families. Classical models are consistent but lower (MCC 0.49–0.53). Among sequence models, BiLSTM+Attention and CNN perform reasonably, while GRU and BiLSTM are weaker. The structural layout of a class (fields, methods, inheritance depth) encoded in the AST is directly relevant to this smell. Precision remains moderate across all families (0.30–0.55), meaning roughly one false positive for every true positive.

8.3.4. Data class

Data Class (312 positives, 7.1%) follows a similar pattern to Blob. GraphSAGE again achieves the highest MCC, followed by GCN and GAT. Among sequence models, CNN, GRU and CodeBERT perform similarly. Classical models range widely, with XGBoost at the top and Naive Bayes at the bottom. LSTM collapses as on all other smells. The AST captures the structural signal (few methods, many fields, minimal behavior) that characterizes this smell.

8.3.5. Comparison with Madeyski and Lewowski

Madeyski and Lewowski [91] conducted the most directly comparable study, training classical ML models (including Random Forest, SVM, Decision Tree and Naive Bayes) on the same MLCQ dataset for the same four smells using MCC as primary metric. Their best per-smell MCC values are 0.31 for Feature Envy (FDA, DS1), 0.81 for Long Method (RF, DS2), 0.55 for Blob (RF, DS1) and 0.57 for Data Class (RF, DS1).

Several methodological differences separate the two setups. They define two binarization protocols: DS1 ($C2 = \text{minor} \cup \text{major} \cup \text{critical}$, $C1 = \text{none}$) matches our default labeling rule; DS2 applies a stricter threshold ($C2 = \text{major} \cup \text{critical}$, $C1 = \text{none} \cup \text{minor}$), treating *minor*-severity instances as negative. They filter samples per smell, requiring at least three reviewer assessments (e.g. 3337 for Feature Envy in DS1), whereas we keep all 4366 samples uniformly. They apply 10× oversampling of the minority class via bagging, while we use focal loss for DL models and sample-weight adjustment for classical models with post-hoc threshold tuning.

DS2's stricter binarization is arguably more rigorous: by moving *minor*-severity instances to the negative class, it reduces label noise from borderline annotations and allows models to learn from cleaner supervision. Their sample filtering further improves annotation agreement. This likely contributes to their stronger MCC on Long Method under DS2.

A per-smell comparison using MCC (Table 6) shows that our GNN models achieve higher MCC on Blob (GraphSAGE 0.634 vs. their RF 0.55) and Data Class (GraphSAGE 0.665 vs. their RF 0.57). On Long Method, our BiLSTM+Attention (MCC 0.762) approaches their RF under DS2 (MCC 0.81). On Feature Envy, their FDA (MCC 0.31 under DS1) outperforms our best (BiLSTM+Attention 0.278). The pipeline supports their DS1 and DS2 binarization protocols, so that researchers can evaluate the same models under identical label definitions and isolate the effect of the model from the effect of preprocessing.

9. Threats to validity

This section discusses the main threats that may affect the validity of the survey and the exploratory comparative study.

9.1. Construct validity

Construct validity concerns whether the concepts analyzed in this study are accurately represented. No review protocol was pre-registered prior to the study, which is a limitation common to many software engineering surveys. To partially compensate, the search strategy, inclusion and exclusion criteria and data extraction procedures were defined before the search was conducted and are fully described in Section 3. Although the literature search followed PRISMA guidelines

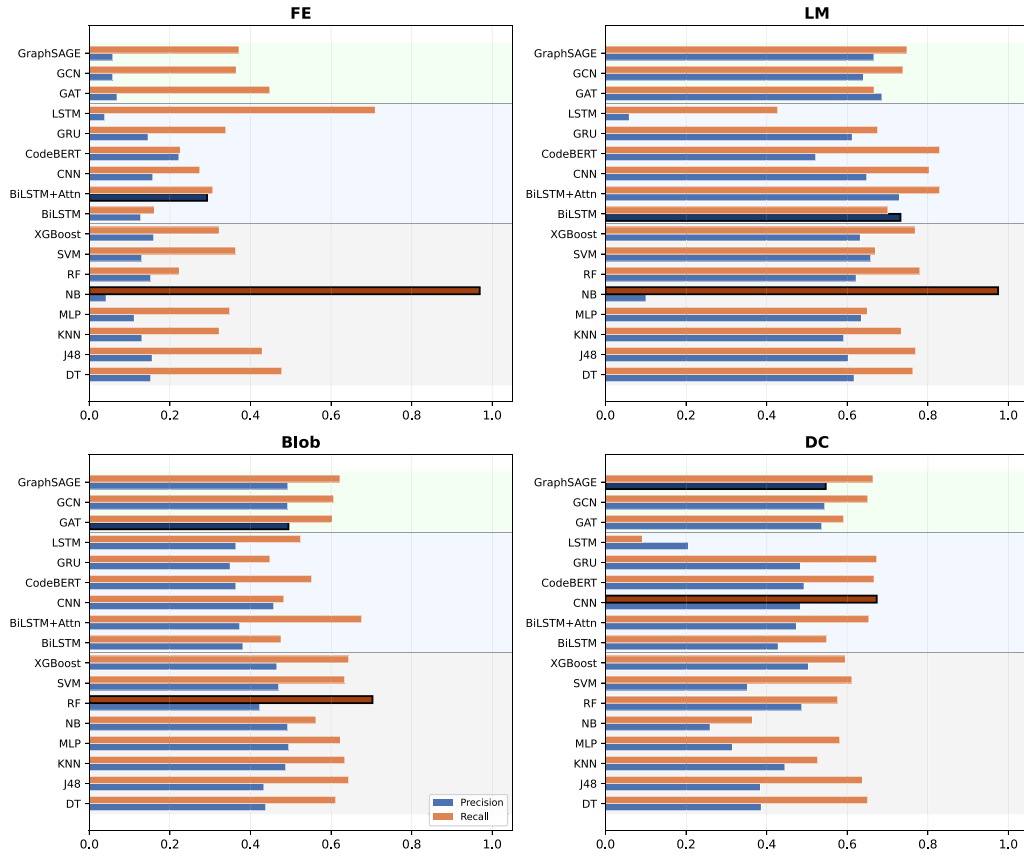


Fig. 9. Precision and recall per smell across all models.

Table 4
Global metrics across model families (MLCQ, 10 seeds).

Family	Model	F1-macro	MCC	PR-AUC
CLASSICAL	XGBoost	.497±.005	.481±.006	.526±.008
	RF	.479±.007	.465±.004	.500±.010
	DT	.477±.006	.464±.007	.437±.018
	J48 (C4.5)	.475±.005	.461±.006	.478±.007
	KNN	.463±.014	.448±.014	.466±.006
	SVM	.457±.010	.441±.010	.450±.005
	MLP	.444±.018	.425±.017	.445±.010
	Naive Bayes	.270±.004	.269±.004	.466±.002
SEQUENCE	BiLSTM+Attn	.522±.035	.506±.033	.535±.035
	CNN	.483±.048	.465±.046	.517±.027
	CodeBERT	.469±.010	.459±.013	.490±.037
	BiLSTM	.435±.039	.413±.045	.446±.050
	GRU	.434±.041	.415±.041	.433±.041
	LSTM	.186±.042	.153±.052	.161±.049
GNN	GCN	.508±.004	.496±.011	.538±.031
	GraphSAGE	.503±.006	.483±.006	.532±.036
	GAT	.495±.007	.480±.009	.538±.051

worst best

Table 5
Per-label F1 scores across model families.

Family	Model	FE	LM	Blob	DC
CLASSICAL	XGBoost	.213±.013	.694±.015	.541±.012	.542±.011
	RF	.175±.027	.691±.012	.524±.018	.526±.013
	DT	.229±.005	.683±.014	.509±.012	.486±.023
	J48 (C4.5)	.226±.009	.677±.013	.518±.010	.478±.027
	KNN	.171±.031	.654±.013	.551±.014	.478±.020
	SVM	.185±.018	.663±.024	.539±.013	.442±.019
	MLP	.173±.028	.642±.016	.551±.007	.411±.046
	Naive Bayes	.080±.000	.182±.001	.524±.012	.294±.005
SEQUENCE	BiLSTM+Attn	.284±.095	.772±.043	.479±.037	.553±.044
	CNN	.179±.062	.715±.064	.466±.116	.572±.069
	CodeBERT	.225±.073	.644±.047	.445±.041	.562±.029
	BiLSTM	.144±.049	.714±.065	.416±.048	.465±.078
	GRU	.157±.097	.635±.082	.378±.129	.567±.048
	LSTM	.060±.031	.133±.151	.426±.031	.127±.033
GNN	GCN	.111±.014	.681±.061	.589±.028	.651±.030
	GraphSAGE	.096±.010	.720±.038	.570±.042	.626±.012
	GAT	.117±.046	.693±.056	.555±.023	.616±.045

worst best

and combined multiple sources (Google Scholar, IEEE Xplore, ACM DL) together with snowballing, it is still possible that relevant studies were missed, especially those published outside mainstream venues or indexed under different terminology (for example design defects, antipattern detection or architecture smells). The inclusion and exclusion criteria, while necessary for consistency, may also have filtered out borderline studies that partially overlapped with smell detection. For the empirical comparison, we relied on the MLCQ dataset; its labels,

like most smell datasets, inherit subjectivity from prior annotations and tool-assisted labeling processes.

9.2. Internal validity

Internal validity relates to the correctness of the data extraction, coding and experimental procedures. All studies were reviewed and coded manually and although metadata was systematically validated,

Table 6
Per-label MCC scores across model families.

Family	Model	FE	LM	Blob	DC
CLASSICAL	XGBoost	.210±.013	.682±.015	.517±.010	.516±.011
	RF	.171±.019	.680±.011	.508±.018	.502±.009
	DT	.239±.007	.673±.010	.488±.013	.457±.024
	J48 (C4.5)	.229±.013	.669±.014	.497±.014	.450±.026
	KNN	.174±.035	.641±.014	.527±.013	.450±.018
	SVM	.186±.014	.649±.024	.516±.012	.411±.021
	MLP	.169±.028	.627±.013	.525±.009	.380±.042
	Naive Bayes	.131±.002	.212±.003	.491±.012	.241±.007
SEQUENCE	BiLSTM+Attn	.278±.085	.762±.044	.457±.032	.526±.050
	CNN	.165±.068	.706±.058	.440±.108	.549±.067
	CodeBERT	.238±.105	.640±.051	.412±.039	.545±.033
	BiLSTM	.137±.058	.700±.066	.377±.050	.438±.078
	GRU	.147±.101	.629±.079	.342±.131	.542±.051
	LSTM	.030±.034	.083±.157	.397±.051	.101±.046
GNN	GCN	.175±.020	.754±.050	.589±.030	.570±.035
	GraphSAGE	.127±.015	.620±.040	.634±.045	.665±.015
	GAT	.172±.050	.740±.055	.562±.025	.568±.050



classification errors are still possible, particularly regarding the precise modeling category (heuristic, ML, DL) and the presence or absence of class-imbalance handling. For the comparative study, we used a uniform preprocessing and training pipeline across models, but alternative hyperparameters or tuning strategies could influence the reported results. Only a subset of representative models was implemented and no extensive hyperparameter optimization was performed, which may bias the relative performance of some techniques.

9.3. External validity

External validity concerns the generalizability of the findings. The survey reflects the current state of research, which is heavily dominated by Java-based datasets and a narrow set of smells (primarily Blob, Long Method, Data Class and Feature Envy). Smells with architectural or cross-component characteristics remain underrepresented in the literature and therefore in this survey. The empirical comparison covers the four smells annotated in MLCQ (Feature Envy, Long Method, Blob and Data Class) but uses a single dataset and a single language (Java), which limits the extent to which the findings can be generalized to other smells, languages or industrial systems.

9.4. Conclusion validity

Conclusion validity relates to whether the interpretations drawn from the results are justified. The exploratory comparative study is not intended as a definitive benchmark and the results should be interpreted as indicative rather than conclusive. Performance differences between models may be influenced by dataset characteristics particularly class imbalance and annotation noise rather than intrinsic modeling capabilities. Additionally, because only a limited number of smells and models were analyzed, the conclusions regarding representation quality (for example metrics vs. tokens) should not be over-generalized without further multi-dataset replication.

9.5. Limitations

This work has limitations. The benchmark covers four smells and three model families (classical ML, sequence DL and GNN) but uses a single dataset (MLCQ) and a single language (Java). Extending the study to additional smells, languages, datasets and model types (e.g. LLM-based prompting) would provide a more complete view.

Overall, these threats do not invalidate the insights presented but outline the boundaries within which the findings should be interpreted. Future work that expands the dataset scope, diversifies smell types and incorporates structural and prompt-based models would help strengthen the generalizability and robustness of the conclusions.

9.6. Implications for researchers and practitioners

Based on the survey findings and the comparative study, we identify several implications for both researchers and practitioners working on code smell detection.

For researchers.

- Standardized benchmarks are needed. The current landscape is fragmented across datasets, label definitions, evaluation metrics and splitting strategies, making cross-study comparison unreliable. Unified experimental pipelines, such as the one proposed in this work, can help establish common ground.
- Multi-annotator agreement should be reported. Datasets built from a single annotator or tool inherit unknown bias. When human labels are used, inter-rater reliability statistics (e.g. Cohen's kappa) should be computed and published alongside the data.
- Class imbalance handling must be explicit. Many studies report high accuracy without acknowledging that the non-smelly majority class dominates. Metrics such as MCC and per-label F1 give a more faithful picture than accuracy alone.
- Reproducibility remains a bottleneck. Fewer than half of the surveyed studies publish their code or data. Systematic publication of replication packages would accelerate progress and enable fair comparison.
- Under-explored smells deserve attention. The vast majority of work targets Blob, Long Method, Feature Envy and Data Class. Architectural smells, test smells and cross-component issues remain largely unaddressed by learning-based methods.

For practitioners.

- Rule-based tools remain a practical starting point. For well-defined smells with clear metric thresholds (e.g. Long Method via LOC, Blob via WMC), heuristic detectors such as PMD or DesigniteJava provide immediate, interpretable feedback without requiring labeled training data.
- ML-based detectors require high-quality labels. Deploying a trained classifier on a new codebase without domain-specific validation risks high false-positive rates, particularly for subjective smells such as Feature Envy.
- Representation choice matters. Our benchmark shows that the same dataset yields substantially different results depending on whether the code is represented as OO metrics, token sequences or AST graphs. Practitioners should consider which representation best captures the structural property targeted by a given smell.
- No single model dominates across all smells. Different model families excel at different smells: GNNs leverage structural context for class-level smells (Blob, Data Class), while sequence models capture naming and token patterns relevant to Long Method. A modular pipeline that allows switching between representations and models per smell is preferable to a one-size-fits-all approach.

10. Conclusion

This survey has reviewed 31 primary studies on automatic code smell detection, covering heuristic tools, evolutionary search, metric-based machine learning, token-based deep learning and recent work with pretrained models. The collected evidence shows a field that has diversified rapidly, yet remains uneven in terms of the types of smells addressed, the programming languages considered and the methodological practices adopted. Most studies focus on a narrow subset of smells,

primarily God Class, Long Method, Feature Envy and Data Class and rely heavily on Java-based open-source systems.

Across the surveyed literature, three recurring issues emerge. First, datasets are fragmented and often insufficiently documented, limiting comparability and raising questions about label quality. Second, class imbalance and subjectivity in smell annotation are widespread, yet often insufficiently handled in evaluation protocols. Third, reproducibility remains limited: only a minority of studies publish complete code or replication packages, making it difficult to validate or extend prior work.

To complement this synthesis, we conducted a comparative study on the MLCQ dataset, evaluating 17 models from three families (classical ML on OO metrics, sequence DL on token sequences and GNNs on AST graphs) under a unified pipeline. The results show that no single family dominates across all four smells: GNNs excel on class-level smells (Blob, Data Class) where structural information encoded in the AST is most relevant, sequence models lead on Long Method where token-level patterns are salient, and Feature Envy remains an open challenge for all families due to the lack of inter-class context in current representations. These findings reinforce the importance of representation choice in smell detection and highlight the need for standardized evaluation settings.

Several directions could extend this work. First, a systematic comparison of imbalance-handling strategies is needed: our pipeline uses focal loss with class-weighted positive weights for DL models and sample-weight adjustment for some classical models, but alternatives such as asymmetric loss, SMOTE-based oversampling or data-level rebalancing (as used by Madeyski and Lewowski) may yield different precision-recall tradeoffs across smells. Second, enriching input representations could improve detection of smells like Feature Envy, for instance by augmenting AST graphs with inter-class dependency edges or combining OO metrics with token-level features in a hybrid model. Third, integrating additional metric extraction tools alongside DesigniteJava would allow measuring how much the choice of feature set affects model rankings independently of the model itself. Fourth, extending the benchmark to additional datasets such as DaCoSX would test whether the patterns observed on MLCQ generalize to other annotation protocols and codebases. Finally, the role of pretrained code models in smell detection is still underexplored: our initial results show that CodeBERT does not outperform a simpler BiLSTM+Attention model, but task-specific fine-tuning strategies and larger-scale pretraining may change this picture.

CRedit authorship contribution statement

Djamel Mesbah: Writing – original draft, Software, Methodology, Data curation, Conceptualization. **Nour El Madhoun:** Writing – review & editing, Writing – original draft, Validation, Supervision. **Khalidoun Al Agha:** Writing – review & editing, Validation, Supervision. **Anis Zouaoui:** Writing – review & editing, Writing – original draft, Supervision, Resources.

Funding

This work was supported by the French Association Nationale de la Recherche et de la Technologie (ANRT) through a CIFRE doctoral fellowship (Convention Industrielle de Formation par la Recherche, grant No.2023/0632), carried out in partnership with Adservio.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Djamel Mesbah reports financial support was provided by Adservio. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The reproduction package (code, configurations and pipeline) is publicly available at <https://github.com/Kheims/DETECT>.

References

- [1] F.A. Fontana, V. Ferme, A. Marino, B. Walter, P. Martenka, Investigating the impact of code smells on system's quality: An empirical study on systems of different application domains, in: 2013 IEEE International Conference on Software Maintenance, IEEE, 2013, pp. 260–269.
- [2] A. Kaur, A systematic literature review on empirical analysis of the relationship between code smells and software quality attributes, *Arch. Comput. Methods Eng.* 27 (4) (2020) 1267–1296.
- [3] G. Hecht, N. Moha, R. Rouvoy, An empirical study of the performance impacts of android code smells, in: *Proceedings of the International Conference on Mobile Software Engineering and Systems*, 2016, pp. 59–69.
- [4] M. Fowler, K. Beck, J. Brant, W. Opdyke, D. Roberts, *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 1999.
- [5] M. Hozano, A. Garcia, B. Fonseca, E. Costa, Are you smelling it? Investigating how similar developers detect code smells, *Inf. Softw. Technol.* 93 (2018) 130–146.
- [6] N. Moha, Y.-G. Guéhéneuc, L. Duchien, A.-F. Le Meur, Decor: A method for the specification and detection of code and design smells, *IEEE Trans. Softw. Eng.* 36 (1) (2009) 20–36.
- [7] N. Tsantalis, T. Chaikalis, A. Chatzigeorgiou, JDeodorant: Identification and removal of type-checking bad smells, in: 2008 12th European Conference on Software Maintenance and Reengineering, IEEE, 2008, pp. 329–331.
- [8] F.A. Fontana, E. Mariani, A. Mornio, R. Sormani, A. Tonello, An experience report on using code smells detection tools, in: 2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops, IEEE, 2011, pp. 450–457.
- [9] D. Di Nucci, F. Palomba, D.A. Tamburri, A. Serebrenik, A. De Lucia, Detecting code smells using machine learning techniques: Are we there yet? in: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (Saner), IEEE, 2018, pp. 612–621.
- [10] D. Mesbah, N. El Madhoun, K. Al Agha, H. Chalouati, Exploring NLP techniques for code smell detection: A comparative study, in: *International Conference on Advanced Information Networking and Applications*, Springer, 2025, pp. 93–104.
- [11] W. Xu, X. Zhang, Multi-Granularity code smell detection using deep learning method based on abstract syntax tree, in: *SEKE*, 2021, pp. 503–509.
- [12] L. Madeyski, T. Lewowski, MLCQ: Industry-relevant code smell data set, in: *Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering*, 2020, pp. 342–347.
- [13] F.A. Fontana, P. Braione, M. Zannoni, Automatic detection of bad smells in code: An experimental assessment, *J. Object Technol.* 11 (2) (2012) 1–38.
- [14] G. Rasool, Z. Arshad, A review of code smell mining techniques, *J. Softw.: Evol. Process.* 27 (11) (2015) 867–895.
- [15] G. Lacerda, F. Petrillo, M. Pimenta, Y.G. Guéhéneuc, Code smells and refactoring: A tertiary systematic review of challenges and observations, *J. Syst. Softw.* 167 (2020) 110610.
- [16] M.I. Azeem, F. Palomba, L. Shi, Q. Wang, Machine learning techniques for code smell detection: A systematic literature review and meta-analysis, *Inf. Softw. Technol.* 108 (2019) 115–138, <http://dx.doi.org/10.1016/j.infsof.2018.12.009>.
- [17] P.S. Yadav, R.S. Rao, A. Mishra, M. Gupta, Machine Learning-Based methods for code smell detection: A survey, *Appl. Sci.* 14 (1414) (2024) 6149, <http://dx.doi.org/10.3390/app14146149>.
- [18] Y. Zhang, C. Ge, H. Liu, K. Zheng, Code smell detection based on supervised learning models: A survey, *Neurocomputing* 565 (2024) 127014, <http://dx.doi.org/10.1016/j.neucom.2023.127014>.
- [19] T. Lewowski, L. Madeyski, How far are we from reproducible research on code smell detection? A systematic literature review, *Inf. Softw. Technol.* 144 (2022) 106783.
- [20] M.J. Page, J.E. McKenzie, P.M. Bossuyt, I. Boutron, T.C. Hoffmann, C.D. Mulrow, L. Shamseer, J.M. Tetzlaff, E.A. Akl, S.E. Brennan, et al., The PRISMA 2020 statement: an updated guideline for reporting systematic reviews, *BMJ* 372 (2021) n71, <http://dx.doi.org/10.1136/bmj.n71>.
- [21] C. Wohlin, Guidelines for snowballing in systematic literature studies and a replication in software engineering, in: *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering*, 2014, pp. 1–10, <http://dx.doi.org/10.1145/2601248.2601268>.
- [22] F.A. Fontana, M. Zannoni, A. Marino, M.V. Mäntylä, Code smell detection: Towards a machine learning-based approach, in: 2013 IEEE International Conference on Software Maintenance, IEEE, 2013, pp. 396–399.
- [23] W. Kessentini, M. Kessentini, H. Sahraoui, S. Bechikh, A. Ouni, A cooperative parallel search-based software engineering approach for code-smells detection, *IEEE Trans. Softw. Eng.* 40 (9) (2014) 841–861.

- [24] F. Palomba, Textual analysis for code smell detection, in: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, 2, IEEE, 2015, pp. 769–771.
- [25] T. Paiva, A. Damasceno, E. Figueiredo, C. Sant’Anna, On the evaluation of code smells and detection tools, *J. Softw. Eng. Res. Dev.* 5 (2017) 1–28.
- [26] D.K. Kim, Finding bad code smells with neural network models, *Int. J. Electr. Comput. Eng.* 7 (6) (2017) 3613.
- [27] G. Rasool, Z. Arshad, A lightweight approach for detection of code smells, *Arab. J. Sci. Eng.* 42 (2017) 483–506.
- [28] A. Kaur, S. Jain, S. Goel, A support vector machine based approach for code smell detection, in: 2017 International Conference on Machine Learning and Data Science, MLDS, IEEE, 2017, pp. 9–14.
- [29] K. Karadžević-Hadžiabdić, R. Spahić, Comparison of machine learning methods for code smell detection using reduced features, in: 2018 3rd International Conference on Computer Science and Engineering, UBMK, IEEE, 2018, pp. 670–672.
- [30] G. Saranya, H.K. Nehemiah, A. Kannan, V. Nithya, Model level code smell detection using egapso based on similarity measures, *Alex. Eng. J.* 57 (3) (2018) 1631–1642.
- [31] B. Bafandeh Mayvan, A. Rasoolzadegan, A. Javan Jafari, Bad smell detection using quality metrics and refactoring opportunities, *J. Softw.: Evol. Process.* 32 (8) (2020) e2255.
- [32] H. Chen, Z. Ren, L. Qiao, Z. Zhou, G. Gao, Y. Ma, H. Jiang, Adaboost-based refused bequest code smell detection with synthetic instances, in: 2020 7th International Conference on Dependable Systems and their Applications, DSA, IEEE, 2020, pp. 78–89.
- [33] T. Sharma, V. Efstathiou, P. Louridas, D. Spinellis, Code smell detection by deep direct-learning and transfer-learning, *J. Syst. Softw. Elsevier* 176 (2021) 110936.
- [34] C. Soomlek, J.N. van Rijn, M.M. Bonsangue, Automatic human-like detection of code smells, in: International Conference on Discovery Science, Springer, 2021, pp. 19–28.
- [35] A. Patnaik, N. Padhy, A hybrid approach to identify code smell using machine learning algorithms, *Int. J. Open Source Softw. Process. (IJOSSP)* 12 (2) (2021) 21–35.
- [36] S. Boutaib, S. Bechikh, F. Palomba, M. Elarbi, M. Makhlof, L.B. Said, Code smell detection and identification in imbalanced environments, *Expert Syst. Appl.* 166 (2021) 114076.
- [37] M.M. Draz, M.S. Farhan, S.N. Abdulkader, M. Gafar, Code smell detection using whale optimization algorithm, *Comput. Mater. Contin.* 68 (2021) 1919–1935.
- [38] Y. Li, X. Zhang, Multi-Label code smell detection with hybrid model based on deep learning., *SEKE* (2022) 42–47.
- [39] N.A.A. Khleel, K. Nehéz, Deep convolutional neural network model for bad code smells detection based on oversampling method, *Indones. J. Electr. Eng. Comput. Sci.* 26 (3) (2022) 1725–1735.
- [40] S. Dewangan, R.S. Rao, A. Mishra, M. Gupta, Code smell detection using ensemble machine learning algorithms, *Appl. Sci.* 12 (20) (2022) 10321.
- [41] S. Dewangan, R.S. Rao, Method-level code smells detection using machine learning models, in: International Conference on Computational Intelligence in Pattern Recognition, Springer, 2022, pp. 77–86.
- [42] R.S. Puja, T. Fatema, N. Akhter, A. Khatun, Prediction of code smell from source code: A hybrid approach, in: 2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD), IEEE, 2023, pp. 315–319.
- [43] N.A.A. Khleel, K. Nehéz, Detection of code smells using machine learning techniques combined with data-balancing methods, *Int. J. Adv. Intell. Inform.* 9 (3) (2023) 402–417.
- [44] W. Ma, Y. Yu, X. Ruan, B. Cai, Pre-trained model based feature envy detection, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR, IEEE, 2023, pp. 430–440.
- [45] T. Mashlach, B. Sotto-Mayor, G. Kaminka, M. Kalech, CLEAN++: Code smells extraction for C++, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR, IEEE, 2023, pp. 441–445.
- [46] M. Škipina, J. Slivka, N. Luburić, A. Kovačević, Automatic detection of feature envy and data class code smells using machine learning, *Expert Syst. Appl.* 243 (2024) 122855.
- [47] N.A.A. Khleel, K. Nehéz, Improving accuracy of code smells detection using machine learning with data balancing techniques, *J. Supercomput.* 80 (14) (2024) 21048–21093.
- [48] R. Lima, D. Pereira, C. Barbosa, L. Leite, D. Baia, B. Fonseca, L. Teixeira, J. Souza, Exploring transfer learning for multilingual software quality: Code smells, bugs, and harmful code, *J. Softw. Eng. Res. Dev.* 13 (1) (2025) 13–207.
- [49] M. Zhang, J. Jia, L.F. Capretz, X. Hou, H. Tan, Graph neural network-based long method and blob code smell detection, *Sci. Comput. Program.* 243 (2025) 103284.
- [50] A. Kovačević, J. Slivka, D. Vidaković, K.-G. Grujić, N. Luburić, S. Prokić, G. Sladić, Automatic detection of Long Method and God Class code smells through neural source code embeddings, *Expert. Syst. Appl.* 204 (2022) 117607, Elsevier.
- [51] Y. Zhang, C. Ge, S. Hong, R. Tian, C. Dong, J. Liu, DeleSmell: code smell detection based on deep learning and latent semantic analysis, *Knowl.-Based Syst.* 255 (2022) 109737, Elsevier.
- [52] F. Khomh, M.D. Penta, Y.-G. Guéhéneuc, G. Antoniol, An exploratory study of the impact of antipatterns on class change-and fault-proneness, *Empir. Softw. Eng.* 17 (2012) 243–275.
- [53] M. Zhang, T. Hall, N. Baddoo, Code bad smells: a review of current knowledge, *J. Softw. Maint. Evol.: Res. Pract.* 23 (3) (2011) 179–202.
- [54] M. Fowler, Refactoring: Improving the Design of Existing Code, Addison-Wesley Professional, 2018.
- [55] G. Suryanarayana, G. Samarthyam, T. Sharma, Refactoring for Software Design Smells: Managing Technical Debt, Morgan Kaufmann, 2014.
- [56] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: 2009 13th European Conference on Software Maintenance and Reengineering, 2009, pp. 255–258, <http://dx.doi.org/10.1109/CSMR.2009.59>.
- [57] M.V. Mäntylä, C. Lassenius, Subjective evaluation of software evolvability using code smells: An empirical study, *Empir. Softw. Eng.* 11 (3) (2006) 395–431, <http://dx.doi.org/10.1007/s10664-006-9002-8>.
- [58] R. Marticorena, C. López, Y. Crespo, Extending a taxonomy of bad code smells with metrics, in: Proceedings of 7th International Workshop on Object-Oriented Reengineering, WOOR, Citeseer, 2006, p. 6.
- [59] M. Abbes, F. Khomh, Y.-G. Gueheneuc, G. Antoniol, An empirical study of the impact of two antipatterns, blob and spaghetti code, on program comprehension, in: 2011 15th European Conference on Software Maintenance and Reengineering, IEEE, 2011, pp. 181–190.
- [60] S. Olbrich, D.S. Cruzes, V. Basili, N. Zazworka, The evolution and impact of code smells: A case study of two open source systems, in: 2009 3rd International Symposium on Empirical Software Engineering and Measurement, IEEE, 2009, pp. 390–400.
- [61] P. Meananetra, S. Rongviriyapanish, T. Apiwatanapong, Using software metrics to select refactoring for long method bad smell, in: The 8th Electrical Engineering/Electronics, Computer, Telecommunications and Information Technology (ECTI) Association of Thailand-Conference 2011, IEEE, 2011, pp. 492–495.
- [62] U. Alon, M. Zilberstein, O. Levy, E. Yahav, Code2vec: Learning distributed representations of code, *Proc. ACM Program. Lang.* 3 (POPL) (2019) 1–29.
- [63] U. Alon, S. Brody, O. Levy, E. Yahav, Code2seq: Generating sequences from structured representations of code, 2018, arXiv preprint arXiv:1808.01400.
- [64] A. Kanade, P. Maniatis, G. Balakrishnan, K. Shi, Learning and evaluating contextual embedding of source code, in: International Conference on Machine Learning, PMLR, 2020, pp. 5110–5121.
- [65] H. Nandani, M. Saad, T. Sharma, Dacos—a manually annotated dataset of code smells, in: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories, MSR, IEEE, 2023, pp. 446–450.
- [66] N. Maneerat, P. Muenchaisri, Bad-smell prediction from software design model using machine learning techniques, in: 2011 Eighth International Joint Conference on Computer Science and Software Engineering, JCSSE, IEEE, 2011, pp. 331–336.
- [67] P. Danphitsanuphan, T. Suwantada, Code smell detecting tool and code smell-structure bug relationship, in: 2012 Spring Congress on Engineering and Technology, IEEE, 2012, pp. 1–5.
- [68] X. Wang, Y. Dang, L. Zhang, D. Zhang, E. Lan, H. Mei, Can I clone this piece of code here? in: Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering, 2012, pp. 170–179.
- [69] T. Sharma, P. Mishra, R. Tiwari, Designite: A software design quality assessment tool, in: Proceedings of the 1st International Workshop on Bringing Architectural Design Thinking Into Developers’ Daily Activities, 2016, pp. 1–4.
- [70] C. Marinescu, R. Marinescu, P. Mihancea, D. Ratiu, R. Wettel, iPlasma: An integrated platform for quality assessment of object-oriented design, in: Proceedings of the 21st IEEE International Conference on Software Maintenance (ICSM 2005), Industrial & Tool Proceedings, Tool Demonstration Track, IEEE Computer Society, 2005, pp. 77–80.
- [71] PMD Contributors, PMD - source code analyzer, 2025, <https://docs.pmd-code.org/latest/>. (Accessed 5 July 2025).
- [72] F. Pecorelli, F. Palomba, D. Di Nucci, A. De Lucia, Comparing heuristic and machine learning approaches for metric-based code smell detection, in: 2019 IEEE/ACM 27th International Conference on Program Comprehension, ICPC, IEEE, 2019, pp. 93–104.
- [73] I. Macia, J. Garcia, D. Popescu, A. Garcia, N. Medvidovic, A. von Staa, Are automatically-detected code anomalies relevant to architectural modularity? An exploratory analysis of evolving systems, in: Proceedings of the 11th Annual International Conference on Aspect-Oriented Software Development, 2012, pp. 167–178.
- [74] C. Marinescu, R. Marinescu, P. Mihancea, D. Ratiu, R. Wettel, iPlasma: An integrated platform for quality assessment of object-oriented design, in: IEEE International Conference on Software Maintenance, Industrial and Tool Volume, 2005, pp. 77–80.
- [75] B.L. Sousa, P.P. Souza, E.M. Fernandes, K.A. Ferreira, M.A. Bigonha, FindSmells: flexible composition of bad smell detection strategies, in: 2017 IEEE/ACM 25th International Conference on Program Comprehension, ICPC, IEEE, 2017, pp. 360–363.
- [76] M. Kessentini, S. Vaucher, H. Sahraoui, Deviance from perfection is a better criterion than closeness to evil when identifying risky code, in: Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering, 2010, pp. 113–122.

- [77] U. Mansoor, M. Kessentini, B.R. Maxim, K. Deb, Multi-objective code-smells detection using good and bad design examples, *Softw. Qual. J.* 25 (2017) 529–552.
- [78] G. Saranya, H.K. Nehemiah, A. Kannan, Hybrid particle swarm optimisation with mutation for code smell detection, *Int. J. Bio-Inspired Comput.* 12 (3) (2018) 186–195.
- [79] S. Hassaine, F. Khomh, Y.-G. Guéhéneuc, S. Hamel, IDS: An immune-inspired approach for the detection of software design smells, in: 2010 Seventh International Conference on the Quality of Information and Communications Technology, IEEE, 2010, pp. 343–348.
- [80] R. Panigrahi, L. Kumar, et al., Application of naïve Bayes classifiers for refactoring prediction at the method level, in: 2020 International Conference on Computer Science, Engineering and Applications, ICCSEA, IEEE, 2020, pp. 1–6.
- [81] S. Jain, A. Saha, Improving performance with hybrid feature selection and ensemble machine learning techniques for code smell detection, *Sci. Comput. Program.* 212 (2021) 102713.
- [82] P.S. Yadav, R.S. Rao, A. Mishra, M. Gupta, Machine learning-based methods for code smell detection: a survey, *Appl. Sci.* 14 (14) (2024) 6149.
- [83] J. Zhang, X. Wang, H. Zhang, H. Sun, K. Wang, X. Liu, A novel neural source code representation based on abstract syntax tree, in: 2019 IEEE/ACM 41st International Conference on Software Engineering, (ICSE), IEEE, 2019, pp. 783–794.
- [84] H. Nanadani, M. Saad, T. Sharma, Calibrating deep learning-based code smell detection using human feedback, in: 2023 IEEE 23rd International Working Conference on Source Code Analysis and Manipulation, SCAM, IEEE, 2023, pp. 37–48.
- [85] A. Kovačević, N. Luburić, J. Slivka, S. Prokić, K.-G. Grujić, D. Vidaković, G. Sladić, Automatic detection of code smells using metrics and CodeT5 embeddings: a case study in c#, *Neural Comput. Appl.* 36 (16) (2024) 9203–9220.
- [86] K. Liu, G. Yang, X. Chen, Y. Zhou, El-codebert: Better exploiting codebert to support source code-related classification tasks, in: Proceedings of the 13th Asia-Pacific Symposium on Internetware, 2022, pp. 147–155.
- [87] D. Mesbah, N. El Madhoun, K. Al Agha, H. Chalouati, Leveraging prompt-based large language models for code smell detection: a comparative study on the MLCQ dataset, in: International Conference on Emerging Internet, Data & Web Technologies, Springer, 2025, pp. 444–454.
- [88] P. Keller, A.K. Kaboré, L. Plein, J. Klein, Y. Le Traon, T.F. Bissyandé, What you see is what it means! semantic representation learning of code based on visualization and transfer learning, *ACM Trans. Softw. Eng. Methodol. (TOSEM)* 31 (2) (2021) 1–34.
- [89] A. Mastropaolo, N. Cooper, D.N. Palacio, S. Scalabrino, D. Poshyvanyk, R. Oliveto, G. Bavota, Using transfer learning for code-related tasks, *IEEE Trans. Softw. Eng.* 49 (4) (2022) 1580–1598.
- [90] J. Yao, M. Shepperd, The impact of using biased performance metrics on software defect prediction research, *Inf. Softw. Technol.* 139 (2021) 106664.
- [91] L. Madeyski, T. Lewowski, Detecting code smells using industry-relevant data, *Inf. Softw. Technol.* 155 (2023) 107112.